



DEPARTMENT OF COMPUTER SCIENCE

Network Analysis of Cryptocurrency Transactions

Alexander Brown

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Master of Engineering in the Faculty of Engineering.

Friday 6th May, 2022

Abstract

This project aims to explore the Bitcoin transaction network, and test whether the global clustering coefficient of the daily transaction network can be used to build a model to predict short-term volatility in the Bitcoin price. To test our hypothesis, we construct two VAR models, one acting as a control with Bitcoin volume and volatility as inputs, and the other one additionally having the global clustering coefficient as input. We use the VAR models to forecast volatility at a horizon of 1 and 7 days, and compare the performances of the two models. We perform our experiments twice, once on the address-level transaction graph, and another time on the user-level transaction graph.

Our main results are summarized as follows. 1. The models that included the clustering coefficient for both the address-level and user-level graphs did not perform significantly better than the control models. Hence, we conclude that there is insufficient evidence that the clustering coefficient is a good predictor of volatility at this time-frame. 2. The clustering coefficient was found to undergo drastic changes in regime during times of bubbles and crashes in the Bitcoin price. We theorize that this result can be used to build a model that can detect the state of the market (bullish or bearish).

For our project, we used public transaction data from the Bitcoin blockchain, provided by Mesonomics. The dataset contained over 1000 GB of data, which was processed and used to build the transaction graphs whose network properties were computed and used in our VAR models.

Dedication and Acknowledgements

I would like to thank Dr. Steve Phelps and Dr. Kiran Kalidindi from Mesonomics for providing me with the idea and data used in this project, as well as for the vast amount of knowledge they shared and helped me acquire during my time working with them.

I would also like to thank my supervisor Dr. John Cartlidge for guiding me through my thesis every step of the way and for providing me with the opportunity to undertake this project.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Alexander Brown, Friday 6th May, 2022

Contents

1	Introduction	1
2	Technical Background	4
2.1	An overview of Bitcoin	4
2.2	The Bitcoin Transaction Network	6
2.3	Network Science	8
2.4	Time Series Analysis	9
3	Literature Review	14
3.1	Address Clustering	14
3.2	Forecasting	14
3.3	Other work	15
4	Project Execution	16
4.1	The Bitcoin Transaction Graph	16
4.2	Bitcoin Price Data	17
4.3	Time Series Variables	17
4.4	VAR Models	19
5	Results	23
5.1	Exploratory Analysis	23
5.2	Regime Change	25
5.3	VAR Models	25
6	Conclusion	30
6.1	Future Work	30
A		35
A.1	Global Clustering Coefficient Algorithm	35
A.2	BAN VAR Output - Prior to May 1 _{st} 2021	35
A.3	BAN VAR Output - After May 1 _{st} 2021	38
A.4	BUN VAR Output	41

List of Figures

2.1	Example transaction. The yellow ovals represent UTXO's belonging to John's public key, which form the inputs to a later transaction initiated by John. The change from the transaction is sent back to John's address.	4
2.2	The structure of the blockchain. Each block holds a hash of the previous block's data, forming a chain of blocks all the way to Block 0, the first block in the chain.	5
2.3	The two graphs represent the same transaction, but the left graph represents the address-level graph, while the right graph represents the user-level graph. Note that not all inputs and outputs in transactions always belong to single entities. The mapping of addresses to users is controlled by a wallet mapping algorithm, which can produce different results based on the heuristics it uses.	6
2.4	Transaction with hash 3281f96f3c458a4bee6248d6667c45e8481f51f4f79b878faedbf2e385dfdb95. The transaction contains 2 inputs and 2 outputs.	7
2.5	The structure of a transaction in the transaction graph.	7
2.6	A histogram representing the degree distribution of nodes of the Internet at the level of autonomous systems (taken from <i>Networks</i> , M. Newman [43]).	8
2.7	An example of an open triplet. If edge <i>ac</i> (dashed line) existed, then the triplet would be considered closed.	9
2.8	A time-series of the Bitcoin price (in US Dollars) for the dates 01/09/2020 up to 01/03/2021.	10
2.9	A graph showing clear seasonality (yearly in this case).	11
2.10	The result of 1 _{st} order differencing at a lag of 4 of the Australian beer production time-series from Fig. 2.9	12
2.11	Part of the output of a VAR(3) using volatility and volume time-series as variables.	12
4.1	Plot of price, volume and the clustering coefficient. Notice the regime change of the clustering coefficient at 2021-5, corresponding with the crash in price.	18
4.2	Summary of the changes to the global clustering coefficient during two identified time periods – before a crash and after a crash.	18
4.3	ACF plot of the global clustering coefficient time-series. Note the peaks every 7 days, indicating weekly seasonality.	19
4.4	Result of seasonal decomposition of the global clustering coefficient time-series using Python's seasonal_decompose . Note the seasonal component repeats 4 times per month, indicating weekly seasonality.	20
4.5	Cross-validation for time-series data. The first iteration (top) contains a train/test split such that the training set contains enough data to be considered reliable. For each subsequent iteration, the previous test set is added to the training set and a new test set is used (image taken from [44]).	21
5.1	Degree distributions of the Bitcoin transaction network	23
5.2	Visualization of block at height 720,000 at the address level.	24
5.3	VAR output for the three-variable model for volatility.	26
5.4	Scatter plots of the residuals from the three-variable VAR model.	27
5.5	RMSE plots for the two models. Horizontal lines represent the means.	27
5.6	RMSE plots for the two models. Horizontal lines represent the means.	28
5.7	Scatter plots of the residuals from the three-variable VAR model.	28
5.8	RMSE plots for the two models. Horizontal lines represent the means.	29

A.1	VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.	36
A.2	CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.	37
A.3	VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.	39
A.4	CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.	40
A.5	VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.	42
A.6	CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.	43

List of Tables

5.1	R^2 scores for the two models measured using in-sample data. The row in bold represents the variable we are interested in - volatility.	26
5.2	DW test scores for each variable. As all values are very close to 2, it shows there is no autocorrelation at lag 1 in the residuals.	27
5.3	DW test scores for each variable in the VAR model.	29
5.4	RMSE scores across tests for the two models. Results in bold indicate better performance.	29

Ethics Statement

This project did not require ethical review, as determined by my supervisor, Dr. John Cartlidge.

Supporting Technologies

- I used Gephi [2] for visualizing the transaction graphs.
- I used Apache Spark and GraphX [8] for the implementation of the graphs and computation of graph metrics.
- Blockchain and transaction data was provided by Mesonomics.
- I used Python and R for statistical analysis with some external libraries, mainly the `statsmodels` [3] library for Python and the `vars` [25] package in R.

Notation and Acronyms

DLT	:	Distributed Ledger Technology
P2P	:	Peer-to-Peer
UTXO	:	Unspent Transaction Output
BTC	:	Bitcoin
ECC	:	Elliptic Curve Cryptography
BAN	:	Bitcoin Address Network
BUN	:	Bitcoin User Network
AR	:	AutoRegression
VAR	:	Vector Autoregression
ACF	:	AutoCorrelation Function
AIC	:	Akaike Information Criterion
BIC	:	Bayesian Information Criterion
ADF	:	Augmented Dickey-Fuller
DW	:	Durbin Watson
RMSE	:	Root Mean Square Error
CPD	:	Change Point Detection

Chapter 1

Introduction

Since its inception in 2009, Bitcoin’s [41] groundbreaking approach to a digital currency implementation has brought about a revolution not only in the world of finance, but also in the world of research, where academics now have access to transparent public transaction data that was never available before for any other financial asset. As of April 2022, the market capitalization of Bitcoin stands at \$750B, with an average of 270,000 transactions per day [17].

Bitcoin is a decentralized digital cash system that works by leveraging the blockchain and Distributed Ledger Technology (DLT)[42]. Similarly to cash, it does not explicitly identify its users, and instead uses public key cryptography for the transfer of funds from one public key to another. The main difference between Bitcoin and existing payment solutions such as card networks (Visa, MasterCard) is that Bitcoin has no central authority, and hence has no single point of attack or failure. It is a global online peer-to-peer (P2P)[45] network with no hierarchy and where all participants share the burden of providing network services. P2P networks are inherently resilient and decentralized, leading to desirable properties for a monetary system, including fault tolerance and censorship resistance. Notable examples of the success of P2P networks include Napster, a file-sharing software released in 1999 that reached over 80 million users at its peak[29], and more recently BitTorrent, which in 2009 accounted for over 50% of all internet traffic.[27]

Following Bitcoin, many other cryptocurrencies with differing use cases have been created, with a large influx of new ones being created in the past 5 years. The second largest cryptocurrency is Ethereum[16], a cryptocurrency with “smart-contract” functionality – a transfer protocol that automatically documents, controls or executes relevant events according to the contract’s terms. As of April 2022, there are over 20,000 cryptocurrencies, with the total market capitalization of all cryptocurrencies at \$1.8T.

In the past decade, network science has also seen adoption in a wide variety of fields, including economics, biology and social research. In finance, it has been shown [46] that by modelling transactions between traders in a continuous double-auction market as a time-varying directed graph, network science can be used on the emerging network to explain the efficiency of the price-discovery process. In this project, we aim to combine network science with the Bitcoin network in order to predict economic variables such as volatility. We do this by using publicly available transaction data from the Bitcoin blockchain to construct a time-varying directed graph, where nodes represent individual wallets or addresses, and edges correspond to transfers of bitcoin between wallets. Using the emerging network, we will compute quantitative network properties, specifically the global clustering coefficient (see Section 2.3.2) and attempt to use this data to predict economic variables such as the volatility of the exchange-rate. More details about the Bitcoin transaction network are provided in Section 2.2.

Using the global clustering coefficient for analysis and forecasting on the Bitcoin network using a VAR model is as far as we know a novel idea. A Google Scholar search on April 28th 2022 showed that most existing research that utilizes the Bitcoin transaction network has the intention of finding ways to de-anonymize entities in the network in order to identify users involved in illegal activities such as money laundering and the commerce of illegal substances, or for the purpose of exploration. The literature that was focused on forecasting economic variables such as price and volatility mainly used traditional financial data such as Open, Close, High, Low, and Volume data[4, 23, 57]. By leveraging the large amounts of data available on the blockchain to build the network, we aim to improve the accuracy of currently available services and produce a model that can more accurately predict important metrics.

It is our belief that we can use the global clustering coefficient in addition to Bitcoin financial data, mainly volume and volatility, to create a Vector Autoregression (VAR) model [54] (see Section 2.4.3) that

can provide short-term forecasting capabilities for volatility. We create three-variable (volatility, volume, clustering coefficient) VAR models for both the address-level transaction graph and user-level transaction graph (see Section 2.2 for difference), and compare their performance to the corresponding two-variable VAR models (volatility, volume) that act as controls.

Moreover, we carry out some exploratory analysis of the Bitcoin network, such as analysing the degree distribution and observing the change in network structure during bubbles and crashes. One of our main finds was that during bubbles (2017-2018 and 2020-2021), the global clustering coefficient is much higher than the proceeding periods where the price crashes. This find was in line with our theories that during bubbles, speculative flipping of an asset occurs at a much higher frequency and leads to certain triadic motifs [46] that increase overall connectedness. Moreover, this was also talked about in [51], mentioning that an increase in connectedness is likely to lead to an increase in price as the transactions become distributed throughout the network and become more attractive to new users.

The main results that we found during our project are:

1. The global clustering coefficient was found to be significant in predicting volatility at certain lags, but after deeper analysis it was concluded that it does not significantly increase the forecasting accuracy of a model that relies solely on volatility and volume as its inputs.
2. The global clustering coefficient undergoes drastic changes in regime depending on the state of the market. This behaviour can potentially be refined in to a model that can make a decision about the current market environment (bull/bear market).
3. The degree distribution of the Bitcoin transaction network resembles that of a power-law, and the network can be considered as being a scale-free network.

Significant fluctuation in the exchange rate of cryptocurrencies and their high volatility, as well as the lack of regulation of their use in most countries has led to their categorization as very high risk, speculative assets with many debating their use and role in the world economy[9, 50, 18]. Therefore, the issue of creating models that can predict important economic variables is both relevant and urgent. An accurate forecasting service that can be used as an early-warning system will be of great use to large institutions such as hedge funds and investment banks, as well as for the scientific community and financial analysts as cryptocurrencies gain mass adoption and become a part of everyday life.

Users of the Bitcoin network are encouraged to not re-use addresses, but rather create new ones every time they want to transact on the network. To create the user-level Bitcoin transaction graph where nodes represent users/wallets, an algorithm needs to be implemented that can cluster addresses belonging to single users and map them onto single nodes. There is no single correct way of creating a user mapping algorithm, and hence the use of one that provides the most accurate view of the transaction network is one of the challenges that exist for projects working on the Bitcoin transaction network. Plenty of research has been done on this matter[39, 32, 48, 49], and heuristics have been proposed that allow researchers to identify different addresses belonging to the same user. More specific details regarding address clustering are discussed in Section 2.2.2. Some of the other challenges we were faced with during our project were constraints in time. The collection and processing of the blockchain transaction data into graphs is a process that would take days or weeks depending on the frequency and time-period of the data, due to the size of the databases we were working on (1,000 GB). Moreover, issues in the user mapping process meant that the user mapping data was only available for use in mid-April, and since the deadline for the project was in the first week of May, not a lot of analysis could be done on the user-mapped data, which we theorize can potentially provide more insightful results than the address-level data.

This thesis is split into 6 sections, which are summarized as follows :

1. Section 1 — Introduction
2. Section 2 — Technical Background. In this section, we introduce the related technical background our project is based on and explain their place in our work.
3. Section 3 – Literature Review. Here we discuss various research and work done relating to the Bitcoin transaction network and Bitcoin more generally, much of which forms the base of our work.
4. Section 4 – Project Execution. In this section, we discuss the methods and techniques we use, from how we collect and process our data to how we build and review our models.
5. Section 5 – Results. We present and analyse our results and draw conclusions.

6. Section 6 – Conclusion. We conclude our thesis, discussing our achievements in regard to our original goals, and we examine future work that could be done using our work as foundation.

In summary, the high-level objective of this project is to leverage blockchain data to build a time-varying directed transaction graph. We will then use that graph to compute network metrics that in turn will be used in an attempt to predict economic variables such as volatility. We aim to do this by:

1. Extracting the raw data from the blockchain and processing it into usable databases.
2. Using the processed data to build the Bitcoin transaction graph as a time-varying directed graph.
3. Computing network properties of the graph as time-series data.
4. Using the data to construct models and perform short-term predictions on volatility.

Throughout this paper to avoid confusion, we will be using *Bitcoin* with a capital B to refer to the Bitcoin protocol and network, and *bitcoin* with a lowercase b when referring to the currency.

Chapter 2

Technical Background

2.1 An overview of Bitcoin

2.1.1 Bitcoin Addresses

A user that wants to use the Bitcoin network to send and receive money first needs to create a set of keys – a private and a public key. The public key could be thought of as a user’s bank account number, and is publicly available for others to see and send money to. The corresponding private key is known only to the user and can be thought of as the password for logging in to the user’s bank account. The private key provides control over the public key and is needed to authorize the spending of bitcoin belonging to that public key. So in short, public keys are used to receive bitcoin, and private keys are used to sign transactions in order to authorize the spending of the bitcoin. The private and public keys are considered a pair because the public key is derived from the private key using a one-way cryptographic function, based on Elliptic Curve Cryptography (ECC)[6]. It is computationally infeasible to derive the private key from the public key.

2.1.2 Bitcoin Transactions

In the Bitcoin network, money is sent as part of a transaction, which can constitute any number of inputs and outputs. Inputs and outputs in the transaction correspond to Bitcoin addresses. The transaction is signed by one or more signatures indicating that the user is authorized to spend the funds referenced in the transaction. Once the transaction is included in the blockchain, the funds allocated to the outputs are available to be spent, forming inputs to a new transaction, extending the chain of ownership and starting a new transaction life cycle.

Until an address that was an output in a transaction spends the bitcoin it receives, it forms part of the pool of Unspent Transaction Outputs (UTXO) tracked by the Bitcoin network. In essence, when a user refers to his bitcoin balance, he is referring to the sum of all UTXO’s locked to his signature. When

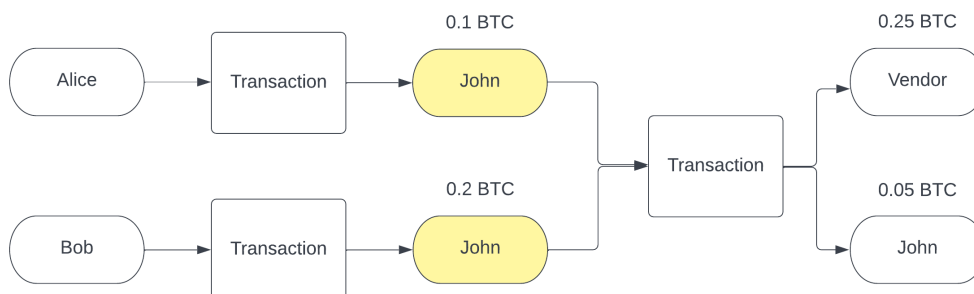


Figure 2.1: Example transaction. The yellow ovals represent UTXO’s belonging to John’s public key, which form the inputs to a later transaction initiated by John. The change from the transaction is sent back to John’s address.

a user wants to spend bitcoin, a number of his UTXO's that add up to the required amount are selected to form the inputs to the transaction (a process usually carried out by the user's wallet software). If the input amount is larger than the transaction amount, a *change address* is provided, and spare change gets sent back to the change address, which can be the original input address or any other address provided by the user.

As an example, suppose Alice sends John 0.1BTC, and Bob sends John 0.2BTC (see Fig. 2.1). John wants to pay for a good that costs 0.25BTC, so his wallet scans the blockchain for UTXO's belonging to John. As the money sent to him by Alice and Bob has not been spent yet, the outputs from those 2 transactions will be identified as 2 different UTXO's (coloured yellow in the example) and used as inputs to the transaction. As John needs change for his transaction, his address will form an output in the transaction where he will receive the change. In reality, there is also a small fee for every transaction which is implied as the difference between the sum of inputs and sum of outputs, but this isn't included in the example for simplicity.

2.1.3 The Bitcoin Network

The term "Bitcoin network" refers to the group of nodes that operate the bitcoin peer-to-peer protocol. The network nodes form a loosely connected mesh with no fixed topology or hierarchy, making all nodes equal peers. Although all nodes are equal, they can take on different roles in the network depending on the network services they are providing. Some nodes run specialized protocols, but nodes can be generally categorized into three main categories, including *full nodes*, *lightweight nodes* and *mining nodes*.

Full nodes maintain a complete, up-to-date copy of the entire blockchain. These nodes are able to independently verify transactions all the way back to the genesis block, and play a vital role in the network as they propagate the blockchain to the entire network to ensure the most trusted version of the blockchain is maintained.

Lightweight nodes have similar functionality to full nodes, however only hold a partial copy of the blockchain that doesn't include transactions. The resulting stored data is 1000 times smaller than the full blockchain, but lightweight nodes cannot construct a full picture of all UTXO's as they do not have data on transactions. They rely on connections to other nodes, including full nodes, to provide their services.

Mining nodes are a special type of node which validate new transactions and record them on the blockchain. New blocks are mined approximately every 10 minutes, containing transactions that occurred since the last block was mined. Transactions that become part of a block and are added to the blockchain are considered confirmed, allowing new owners of bitcoin to spend the bitcoin received in those transactions.

It is important to note that in addition to specialized behaviour, all nodes include basic routing functionality in order to participate in the network, and all nodes participate in the validation of incoming transactions and in turn their propagation to the rest of the network.

2.1.4 The Blockchain

A Block

A block is a data structure that collects and organizes transactions for inclusion in the blockchain. It consists of a metadata-containing header and a long list of transactions in the data body of the block.

A reference to the previous block hash is included in the block header, connecting this block to the prior block in the blockchain. Other information such as the timestamp and difficulty of the block is also included in the header. When referring to specific blocks in the blockchain, instead of using the block's

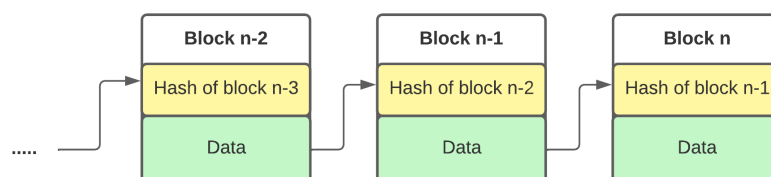


Figure 2.2: The structure of the blockchain. Each block holds a hash of the previous block's data, forming a chain of blocks all the way to Block 0, the first block in the chain.

hash, it is common to use the blocks' "height", which is the block's position in the long list of blocks all the way to the first block. So for example the first block in the blockchain has height 0, the second block has height 1 etc.

The Blockchain

The blockchain data structure is a back-linked, ordered list of transaction blocks. Each block holds a reference to the previous block in the chain. When a transaction occurs, it needs to be delivered to the bitcoin network, so it can be included in the blockchain. The P2P network structure of Bitcoin means that each node is connected to a few other nodes, which in turn are connected to a few other nodes themselves. When a transaction is created, it is sent through the network to its neighbouring nodes and is quickly propagated throughout the whole network in an exponentially expanding ripple until all connected nodes have received it.

Some receiving nodes will be mining nodes, which will aggregate all received transactions into a candidate block. When a candidate block is successfully mined, it is added to the blockchain, confirming all transactions present in the block and allowing the money in those transactions to be spent.

2.2 The Bitcoin Transaction Network

2.2.1 The BAN and BUN

The Bitcoin transaction network is a graph of nodes and edges, where nodes represent addresses (or users), and edges represent transfers of bitcoin between addresses. There are two ways to represent the transaction network, and we will refer to them in this paper as the Bitcoin Address Network (BAN) and Bitcoin User Network (BUN). The difference between the two is that the BAN uses the public addresses as nodes in the network, whereas the BUN uses user wallets as nodes. The BAN is a simpler representation of the network as it only uses publicly available transaction data found on the blockchain, whereas the BUN requires some sort of wallet mapping functionality to map addresses to users. For example, suppose a transaction is made up of 3 inputs and 2 outputs (Fig. 2.3 a)). A wallet mapping algorithm which identifies all 3 inputs as belonging to a single user, and both outputs also belonging to a single user, would result in a simpler graph (Fig 2.3 b)) showing an explicit relationship between the 2 users.

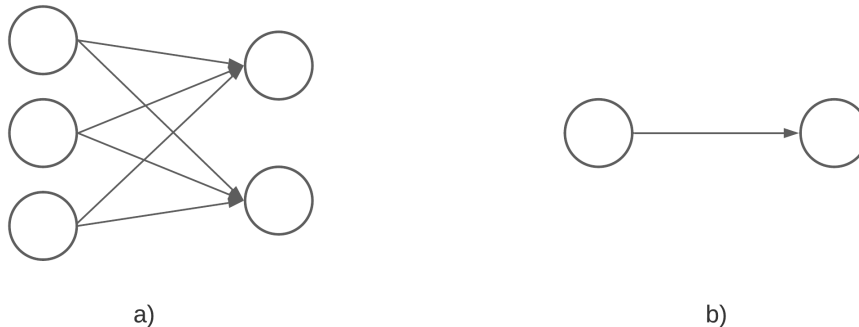


Figure 2.3: The two graphs represent the same transaction, but the left graph represents the address-level graph, while the right graph represents the user-level graph. Note that not all inputs and outputs in transactions always belong to single entities. The mapping of addresses to users is controlled by a wallet mapping algorithm, which can produce different results based on the heuristics it uses.

For a more concrete example of the structure of the transaction graph, we take a real example from the Bitcoin blockchain. Fig. 2.4 shows a transaction^[11] from block with height 700,000.

For the address level graph, there are two predominant ways of representing a transaction. One method is using a bipartite approach, where there are nodes representing transactions as well as nodes representing inputs and outputs. The transaction node will be in the middle of the inputs and outputs, and inputs and outputs will not be directly connected together. However, there are limited meaningful graph metrics that can be computed for such a graph, so for our approach we decided to go with the Cartesian product of the inputs and outputs, i.e., connecting every input to every output. Fig. 2.5 shows

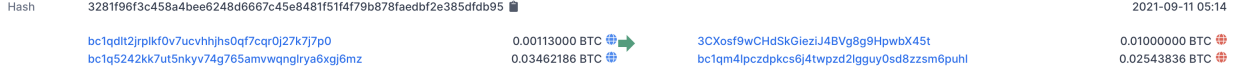


Figure 2.4: Transaction with hash 3281f96f3c458a4bee6248d6667c45e8481f51f4f79b878faedbf2e385dfdb95. The transaction contains 2 inputs and 2 outputs.

how the transaction from Fig. 2.4 would be built for the BAN, where i_1, i_2 represent the inputs and o_1, o_2 represent the outputs.

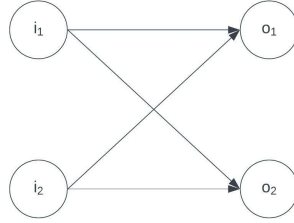


Figure 2.5: The structure of a transaction in the transaction graph.

2.2.2 Address Clustering

In this project, we aim to make use of both the BAN and BUN for our analysis. To construct the BUN, we need an algorithm that will choose whether to map a certain address to a user based on some heuristics. A lot of literature has been published on these heuristics and how best to use them when constructing a transaction graph.

The data and algorithms to construct the BUN were not created by myself, but instead were provided by Mesonomics. As such, I do not have definitive insight in to how exactly the wallet mapping algorithm works, but I know of some core heuristics that are often used:

1. Multi-Input Heuristic

This heuristic has been used many times in research [39, 32, 48, 49] and forms a staple for any address clustering algorithm. It is used to link together inputs that are used in the same transaction. Therefore, we state that if two or more addresses are used together as inputs to a transaction, they are controlled by the same user.

$$\forall pk \in inputs(t), pk \text{ belong to the same user}$$

where pk correspond to public keys, and t is a transaction. The results of this heuristic are transitive and extend beyond the inputs to a single transaction. For example, if we saw one transaction with addresses A and B as inputs and another with addresses B and C as inputs, we would assume that A, B, and C all belonged to the same user.

2. Change Address Heuristic

This heuristic was discussed and described in [39] and focuses on the role of change addresses in the Bitcoin network. Recall that a change address is an address which is part of a transaction's outputs, where spare change from that transaction is sent to. Very often, change addresses are created internally by wallet software and never re-used (often the user of a wallet has no idea about the existence of this address and hence is unlikely to give out this address to other users). However, as this heuristic relies on this idiom of use instead of relying on an inherent property of the Bitcoin protocol (like Heuristic 1), it is less robust and more likely to produce false positives. Therefore, it is important when using this heuristic to design it as safely as possible. One such way of using this heuristic is described below:

1. $d^+(pk) = 1$, i.e., this is the first time pk appears in a transaction.
2. There are no $pk' \in outputs(t)$ such that $pk' \in inputs(t)$ i.e., there isn't a self-change address in the transaction.
3. There are no $pk' \in outputs(t)$ such that $pk \neq pk'$ and $d^+(pk') = 1$ i.e., condition 1 is only met for pk .

where d^+ is the in-degree, pk , pk' correspond to public keys and t is a transaction. Upon meeting the conditions above, a public key pk can be said to be a one-time change address for a transaction t , and is controlled by the same user as the input addresses.

2.3 Network Science

Throughout this paper, we will be making use of network science to explore and analyse the Bitcoin transaction network. In this section, we will introduce some basic network properties that we make use of throughout our project.

2.3.1 Degree Distribution

The *degree* of a node in an undirected graph is the number of edges connected to it. In a directed graph such as the Bitcoin transaction network, each node has two degrees: the *in-degree*, which is the number of edges going into the node, and the *out-degree*, which is the number of edges leaving the node. By obtaining information about each node's in-degree and out-degree, we can obtain a full-scale picture of the network's structure with regard to the degree distribution, which is typically visualized as a graph with frequency or proportion on the y-axis and degree on the x-axis. An example is shown in Fig. 2.6.

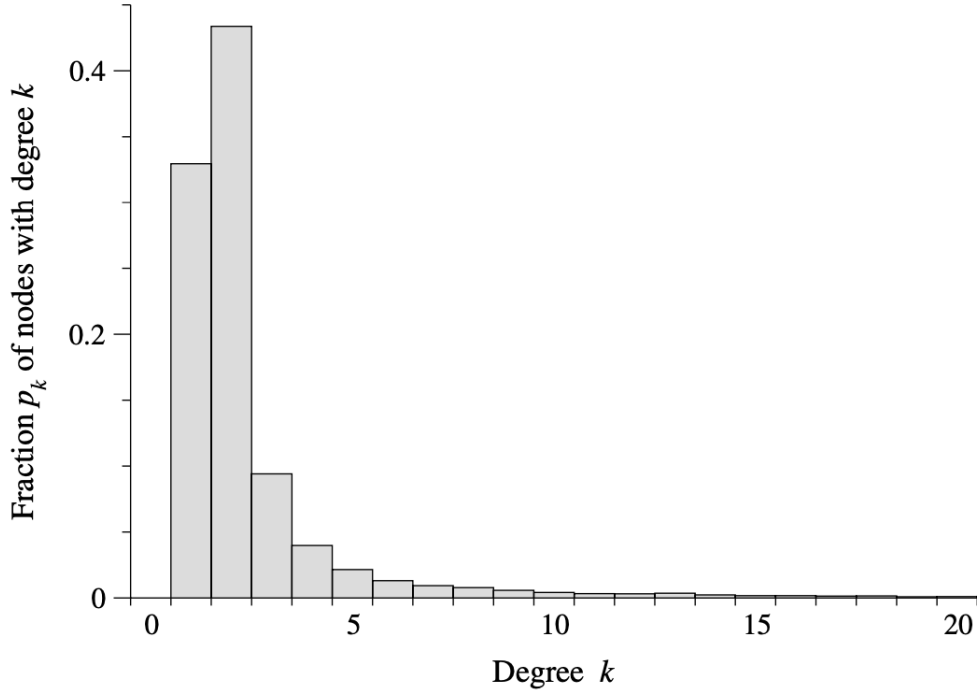


Figure 2.6: A histogram representing the degree distribution of nodes of the Internet at the level of autonomous systems (taken from *Networks*, M. Newman [43]).

2.3.2 Clustering Coefficient

Local Clustering Coefficient

The local clustering coefficient of a node in a network measures the proportion of that nodes' neighbours that are connected with each other. For a directed graph such as the Bitcoin transaction network, the local clustering coefficient of a node i is given by:

$$C_i = \frac{|\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}|}{k_i(k_i - 1)}$$

where E is the set of all edges, $N_i = \{v_j : e_{ij} \in E \wedge e_{ji} \in E\}$ is the neighbourhood of node i , and k_i is the degree of node i .

Global Clustering Coefficient

The global clustering coefficient of a network is a measure of the network's connectedness and density. The method for computing the global clustering coefficient is based on *triplets*. A *triplet* in a network is a set of three nodes that have two or three edges between them. If there are three edges, it is called a *closed triplet*, otherwise it is an *open triplet* (see Fig. 2.7 for example). The global clustering coefficient is defined as the total number of closed triplets over the total number of triplets (open or closed).

$$\text{Global Clustering Coefficient} = \frac{\# \text{ of closed triplets}}{\text{total } \# \text{ of triplets}}$$

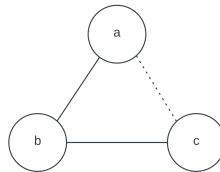


Figure 2.7: An example of an open triplet. If edge ac (dashed line) existed, then the triplet would be considered closed.

An alternative to the global clustering coefficient that is calculated using triplets is the network average clustering coefficient, which is simply the average value of the local clustering coefficient of all nodes in the network:

$$C = \frac{1}{n} \sum_{i=1}^n C_i$$

In this project, we will be using the global clustering coefficient instead of the network average clustering coefficient, as it was more efficient to compute using Spark GraphX.

2.4 Time Series Analysis

2.4.1 Overview

A time-series is a sequence of data points that occur in chronological order over a period of time. Throughout the project, we will be using different time-series data computed from the Bitcoin transaction graph and from Bitcoin price data for our analysis. Fig. 2.8 shows an example time-series.

2.4.2 Stationarity and Seasonality

Stationarity

A *stationary* time-series is one whose properties do not depend on the time at which they are observed. More specifically, a stationary time-series has the property that its mean, variance, and autocorrelation structure does not change over time. Most of the time, analysis on time-series data depends on the data being stationary, otherwise patterns and relationships are often found when there are in fact none. Therefore, stationarity has become a common assumption for many practices and tools in time-series analysis.

Seasonality

Seasonality is another characteristic of time-series where the data undergoes regular and predictable variations every identified time period (see Fig. 2.9 for example, dataset taken from the `fpp` R package [24] and contains quarterly information about Australian beer production). For example, a time series of heating costs in a house will have a yearly seasonality element, as costs will predictably increase in winter and decrease in summer. This is a pattern that will repeat every year, and the data will be said to have yearly seasonality. Seasonality can also come in different forms, such as weekly or daily seasonality, for example volume data in equity markets will tend to have predictable increases in volume at important times such as the opening times of the NYSE and LSE, leading to daily seasonality.

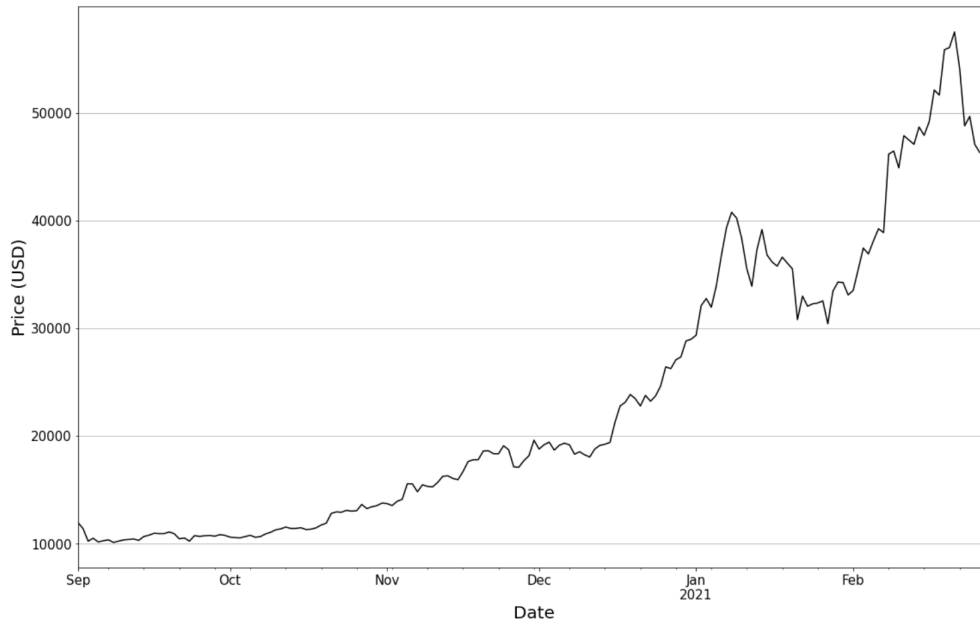


Figure 2.8: A time-series of the Bitcoin price (in US Dollars) for the dates 01/09/2020 up to 01/03/2021.

Similar to stationarity, a time-series that has been processed to remove seasonality is usually a prerequisite when working with time-series.

Testing for stationarity

The method used to test for stationarity throughout our project is the Augmented Dickey-Fuller (ADF) test. The ADF test, tests the null hypothesis that a unit root is present in time-series data. The alternative hypothesis implies stationarity of the underlying time-series data, therefore the ADF test is often used to test whether a time-series is stationary or not. The ADF statistic produced by this test is a negative number; the more negative it is, the stronger the rejection of the null hypothesis.

Removing seasonality and non-stationarity

If a time-series is found to be non-stationary and/or having seasonal components, then techniques can be implemented to try to convert it to a stationary time-series. One such technique is *differencing*, which computes the difference between consecutive observations. Occasionally the differenced data may not appear to be stationary, so it may be necessary to apply the procedure more than once, giving rise to 1st difference, 2nd difference and so on. The first difference of a time-series x is described by the following equation:

$$x_t^1 = x_t - x_{t-1}$$

The general equation for the differences of order d is given by the recursive formula:

$$x_t^d = x_t^{d-1} - x_{t-1}^{d-1}$$

Differencing can also be performed at different *lags*. For example, if a time-series with daily data was found to have weekly seasonality, then to remove that seasonality a difference with lag 7 could be computed, which instead of computing the difference between consecutive observations, would compute the difference between observations 7 steps or lags behind.

Using our example from Fig. 2.9, the ADF test produces a *p-value* of 0.95, meaning there is insufficient evidence to reject the null hypothesis that the time series is stationary. As the data is quarterly, and the seasonality appears to be yearly, then differencing at a lag of 4 would make sense in order to remove the seasonal element.

Fig. 2.10 shows the resulting time-series after differencing one time. The ADF test on the differenced time-series produces a *p-value* of 0.01 and hence the time-series is now stationary.

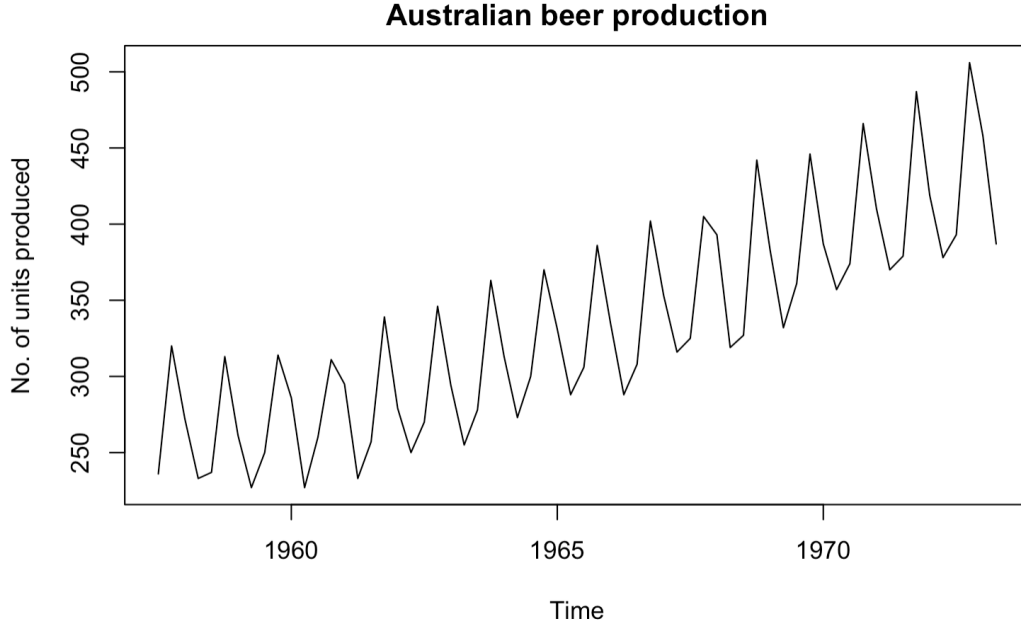


Figure 2.9: A graph showing clear seasonality (yearly in this case).

2.4.3 Autoregressive Models

AR Model

An autoregressive (AR) model is a statistical model which uses past observations to predict future values. It specifies that the output variable is linearly dependent on its own previous values. The previous values are commonly referred to as *lags*. A model of order p contains the last p lags in its calculations. For example, $AR(p)$ for a variable x :

$$x_t = \varepsilon_t + \sum_{i=1}^p \phi_i x_{t-i} + c$$

where $\phi_i, \dots, \phi_p$ are the model parameters, c is a constant and ε_t is white noise.

VAR Model

When the relationship between two or more time-series is to be examined, a Vector Autoregression (VAR) model can be used instead, which is a statistical model that captures the changing relationship between various quantities over time. By allowing for multivariate time series, VAR models extend the single-variable (univariate) AR model. Each variable (time-series) in a VAR model is represented as a linear combination of its own past values and the past values of other variables in the system. As there are multiple time-series that influence each other, a VAR model is a system of equations with one equation per time-series. Similarly to AR models, a VAR model of order p uses p lags. For example, suppose we have two time-series variables, x_1 and x_2 . The VAR(1) model is as follows:

$$\begin{aligned} x_{t,1} &= \varepsilon_{t,1} + \phi_{1,1}x_{t-1,1} + \phi_{1,2}x_{t-1,2} + c_1 \\ x_{t,2} &= \varepsilon_{t,2} + \phi_{2,1}x_{t-1,1} + \phi_{2,2}x_{t-1,2} + c_2 \end{aligned}$$

where ϕ are the model parameters, c are constants and ε is white noise. Each variable is a linear function of the lag 1 values for all variables in the set. A VAR(n) model would include additional variables from previous lags up to lag n .

In a VAR model, there are two choices to make, mainly the number of variables K , and the number of lags p . The number of coefficients that are estimated in a VAR model is equal to $K + pK^2$, and the more coefficients that need to be estimated, the greater the estimation error.

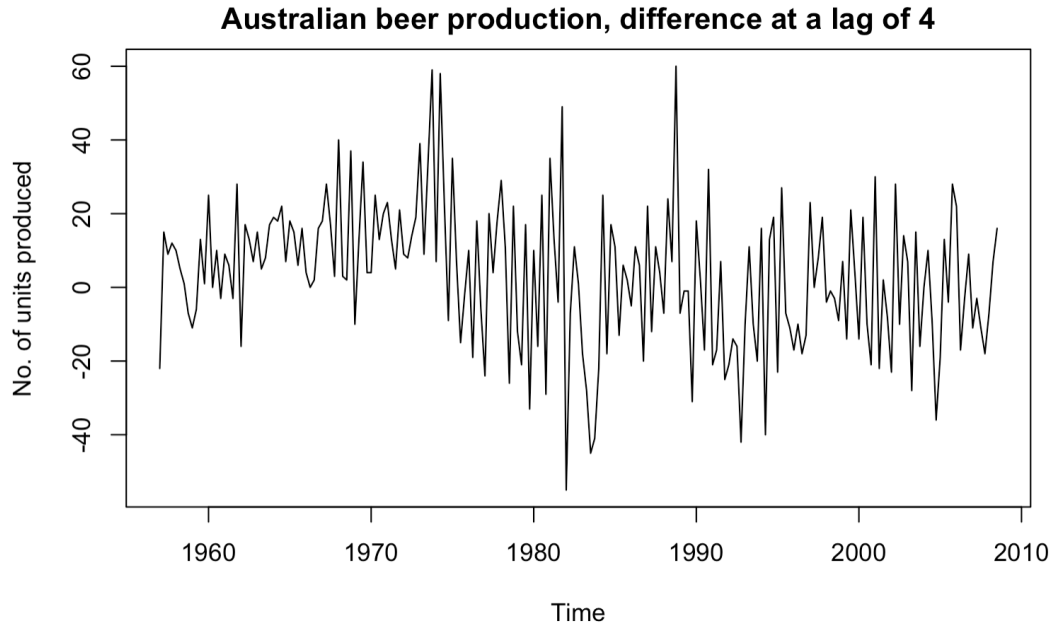


Figure 2.10: The result of 1st order differencing at a lag of 4 of the Australian beer production time-series from Fig. 2.9

Therefore, in practice, it is common to keep K as low as possible by only selecting variables that are useful in forecasting each other. The *Granger Causality* test can be used for this. For the lag length, statistical information criteria are used to identify the ideal lag. This means the VAR model is computed for different lags and certain statistics such as the Bayesian Information Criterion (BIC) and Akaike Information Criterion (AIC) are calculated. The optimal lag is the one that minimizes these statistics. The resulting VAR model can then be used for analysing the relationship between the variables and for forecasting.

In this project, we made use of two VAR models. The first was a control and included two variables, namely volatility and volume, while the second model also included the global clustering coefficient data to see if it could produce a model that could outperform the control.

Interpreting VAR model results

The resulting VAR analysis will output a table for each variable in the model along with all the variables of each lag that make up it's equation and their coefficients, as well as statistics and *p-values* for each variable. As it can be overwhelming to understand the output of a VAR model, I will use an example here to explain it's meaning. The example will be a VAR(3) model using volume and volatility time series data for Bitcoin, and the output will be from R using the `vars` package [25].

```
Estimation results for equation Volatility:
=====
Volatility = Volume.l1 + Volatility.l1 + Volume.l2 + Volatility.l2 + Volume.l3 + Volatility.l3 + const

      Estimate Std. Error t value Pr(>|t|)
Volume.l1    2.1235239   2.1667868    0.980  0.32819
Volatility.l1 -0.8522357   0.0676749  -12.593 < 2e-16 ***
Volume.l2     0.3454507   2.3131116    0.149  0.88142
Volatility.l2 -0.5609482   0.0808999   -6.934 4.91e-11 ***
Volume.l3     3.1228654   2.1374188    1.461  0.14549
Volatility.l3 -0.2179376   0.0681909   -3.196  0.00161 **
const         0.0008491   0.0028289    0.300  0.76435
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Figure 2.11: Part of the output of a VAR(3) using volatility and volume time-series as variables.

Fig. 2.11 shows a snippet of the output for a two-variable VAR(3) model using volatility and volume.

The snippet shows the output for the volatility variable.

The *Estimate* column lists the coefficients for all the variables that make up the volatility equation. The .l1, .l2 and .l3 attached to the variables indicates the lag for which they are referring to. The far right column lists the *p-value* associated with each variable. If the *p-value* is below a critical level, then it is said that the variable it represents is significant in predicting the variable of the equation at that lag. If the *p-value* is not below a critical level, then the coefficient of that variable can be assumed to be 0. In our example, we can see that Volatility.l1, Volatility.l2 and Volatility.l3 are all significant in predicting Volatility. Hence, the equation for Volatility resulting from our model will be:

$$\sigma = -0.85\sigma.l1 - 0.56\sigma.l2 - 0.22\sigma.l3$$

where σ denotes volatility. The same kind of output will exist for all variables of the VAR model, and a system of linear equations relating all the variables is then created.

As well as variable and coefficient values, a VAR model will usually provide other information such as the R^2 values for each variable (quantifying how good the in-sample fit for the variable is), as well as the residuals from the model which can give important information about the relationships between the variables.

Chapter 3

Literature Review

3.1 Address Clustering

Address clustering is one of the fundamental steps included in many high-level blockchain analyses, with the aim of de-anonymisation and tracking of entities in the network. Many heuristics and clustering methods have been proposed and analysed in the past decade, including the multi-input heuristic, change-address heuristics, and transaction fingerprinting. The multi-input heuristic explained in Section 2.2.2 was used many times in previous work [39, 32, 48, 49], and the change-address heuristic was introduced and explained in [39]. These works focused on using these heuristics to cluster and in turn identify entities on the Bitcoin network, in order to track and identify users partaking in illegal activity such as money laundering and theft.

Research done in [30] concluded that by just using the multi-input heuristic, 69% of addresses can be identified. Although still useful, the reliability of the multi-input heuristic in the past years has decreased due to the introduction of services such as *CoinJoin* [1], which is a method of mixing coins to protect user's privacy on the blockchain.

Spagnuolo et al. [52] interestingly created BitIodine, a method that parses blockchain data, clusters addresses likely belonging to the same entity and then classifies and labels them in a semi-automatic manner using information on their actions. BitIodine was successfully used to investigate the *CryptoLocker* [10] ransomware, accurately identifying and quantifying information about the number of ransoms paid, as well as information about victims.

More recently, [40] analysed the performance of the *change-address* heuristic – one of the harder heuristics to get right – in various academic papers and applications, and found that this heuristic is applied inconsistently across studies, many forgoing its use altogether. They present findings on the performance of the change-address heuristic on a dataset where the ground truth is known, and after recommending improvements in the methodology and application of this heuristic, conclude that its inclusion in address clustering algorithms is feasible with high precision.

3.2 Forecasting

Although not specifically using transaction network properties as inputs, a lot of work has been done in the past years regarding forecasting the Bitcoin exchange rate using autoregressive models.

In 2017, Bakar and Rosbi [4] found that ARIMA is a reliable forecasting model for Bitcoin. Their results from using an ARIMA(2,1,2) forecast model found an R-squared value of 0.44, indicating that the model explains 44.44% of the data's variability around its mean. The Mean Absolute Percentage Error (MAPE) of their Bitcoin price forecasts to the actual data was 5.36%.

In 2018, Sutiksno et al. [55] compared forecasts between ARIMA, and a new forecasting method, the α -Sutte Indicator [5]. The results found that the α -Sutte Indicator resulted in better accuracy over the ARIMA model (comparing MAE and MSE values).

In regard to using VAR models, Koutmos [34] developed two separate bivariate VAR models based on percent changes in Bitcoin returns and percent changes in the total number of unique Bitcoin transactions. Results showed that the Bitcoin return value was strongly correlated to its transaction activity.

McNally [38] in 2018 compared the forecasting accuracy of a Recurrent Neural Network (RNN) and Long Short Term Memory (LSTM) network. It was found that the LSTM achieved the better performance, with an accuracy of 52% and RMSE of 8%. The paper also used an ARIMA model as a control

for comparison. As expected, the non-linear machine learning models both outperformed the ARIMA model, which performed poorly.

Similarly to McNally, Glenski et al. [28] built and evaluated LSTM neural networks using historical price data as well as social data. Two LSTM's were compared to each other, one using only price data, with the second one including social data as input. Baseline ARIMA models were used as controls for comparison. The results found that using social data reduced the forecasting error, with the LSTM's outperforming the corresponding ARIMA models.

In 2019, Bush et al. [15] compared many models for forecasting Ethereum STORJ prices. These models included ARIMA, ARMA-GARCH, VAR, α -Sutte, VAR and NNAR. It was found that the VAR model outperformed all others in short and midterm forecasting horizons (5 and 20 days), while the NNAR outperformed the other models in the long-term horizon (50 days).

Phelps and Musial-Gabrys [46] showed that by modelling transactions between agents in a double-auction market for a risky asset as a time-varying directed graph, similarly to what we are doing in this project, and using ideas stemming from network science, namely network motif analysis, they can better explain the efficiency of the market during price discovery.

3.3 Other work

Interesting work has been done in regard to finding how the traditional markets affect Bitcoin. Conrad et al. [20] found that the US stock market and the global stock market index had no significant influence on bitcoin's volatility.

Liu and Tsyvinski [36] showed that macroeconomic factors that usually determine the dynamics of traditional markets such as currencies, stock and commodities markets have no significant effect on the dynamics of the cryptocurrency markets.

Garcia et al. [22] wanted to analyze the effect of various social signals such as information search, word of mouth volume, emotional valence and opinion polarization extracted from tweets relating to Bitcoin. Their results showed that rising Bitcoin prices are preceded by increases in opinion polarization and exchange volume, and that emotional valence precedes rising exchange volumes and opinion polarization. Their results were then used to create algorithmic trading strategies, which showed large profits (over 100%) after some months of trading.

Similarly, Jain et al. [31] and more recently [33] found that cryptocurrency prices, unlike traditional assets, are affected by factors relating to social media, such as the number of Google trend searches and posts on social media platforms such as Twitter.

In 2020, Conlon et al. [19] analysed the performance of Bitcoin during the brief Covid-19 bear market. They aimed to test the "safe haven" status that Bitcoin has achieved during the past few years due to its exceptional performance when compared to more traditional financial assets. Their findings suggested that Bitcoin does not act as a safe haven after all, and some of their results included the fact that even a small allocation of a portfolio in Bitcoin substantially increases downside risk. They conclude that their findings cast doubt on the ability of Bitcoin to provide shelter from turbulence in traditional financial markets.

Chapter 4

Project Execution

The general plan of execution was to obtain a time-series of the global clustering coefficient, and construct VAR models using said time-series in addition to Bitcoin financial data such as volume and volatility to see if there is a relationship between the variables, and importantly if the network properties provide forecasting power. More specifically, we wanted to find out if the global clustering coefficient can be used as a predictor of volatility. We aimed to perform the analysis on both the BAN and BUN, using a VAR model with only volume and volatility as our control, and comparing its performances with the VAR model also containing the global clustering coefficient. Our hope was that by adding the global clustering coefficient, we could achieve significantly better performance. A high-level overview of our method was:

1. Construct the Bitcoin transaction graphs for the days in our required time-period
2. Use the graphs to compute the global clustering coefficient and compile it into a time series
3. Construct two VAR models; a control using only volume and volatility, and one that also contains the global clustering coefficient data.
4. Analyse resulting VAR models

The time-period of our analysis using the BAN is from September 1_{st} 2020 up to August 31_{st} 2021, a period that contained a large increase in the price of bitcoin (500%), followed by a large crash (60%).

For the BUN, our analysis run from May 1_{st} 2021 up to December 31_{st} 2021 as that is the period for which user mapped data was available for use.

4.1 The Bitcoin Transaction Graph

4.1.1 Constructing the Transaction Graph

The initial part of the project was to build the Bitcoin transaction graph. This involved processing raw blockchain data into databases containing information about blocks and transactions within those blocks. The transaction graph was built at the daily level for all the days in our time-period. To build the daily graphs, a query to select all the blocks that were mined in a specific day was used. Using information about the transactions in those blocks, a file of edges was created that contained connections between inputs and outputs for all transactions in that day, where the nodes corresponded to the addresses of said inputs and outputs. After constructing the edge files for all the days in the desired period, they were loaded into Apache Spark GraphX [8] and constructed into graphs.

The collection and processing of transaction data from the blockchain was the most time-consuming part of the project. Even when working remotely on the machine that hosted the databases, processing and creating the edge files for a single block would take on average 20 seconds. Hence, in order to create the edge files for a period of 1 year at the daily level (in a typical day roughly 145 blocks are mined), data collection would run for nearly two weeks. It was therefore very important to get the processing and edge file algorithm correct from the beginning so that data could start to be collected early. This proved to be a challenge early on as after running the data collection algorithm for two weeks, a mistake in the implementation was found making the data redundant.

4.1.2 Computing Network Properties

Apache Spark GraphX is a Spark API for graphs and graph-parallel computation. It comes with functions to build graphs from edge files, which is what we used to construct our graphs. It also comes with built-in functions to compute important metrics such as node degrees, as well as algorithms such as Page Rank and Triangle Count. The latter is an algorithm that we used in order to calculate the global clustering coefficient. As the global clustering coefficient is not a built-in function of GraphX, we used the algorithm shown below to compute it (See Appendix A.1 for explanation of the algorithm):

```
def clusteringCoefficient[VD:ClassTag,ED:ClassTag](g:Graph[VD,ED]) = {
  val numTriplets = g.aggregateMessages[Set[VertexId]](
    et => { et.sendToSrc(Set(et.dstId));
            et.sendToDst(Set(et.srcId)) },
    (a,b) => a ++ b)
    .map(x => {val s = (x._2 - x._1).size; s*(s-1) / 2})
    .reduce(_ + _)
  if (numTriplets == 0) 0.0 else
    g.triangleCount.vertices.map(_._2).reduce(_ + _) /
    numTriplets.toFloat
}
```

Listing 4.1: Function to compute the global clustering coefficient of a graph in Apache Spark. Language is Scala (taken from *Spark GraphX in Action*, M. Malak, R. East) [37]

Initially, inspection of the computed values using this algorithm showed that something wasn't quite right. More specifically, on certain days, the computed clustering coefficient was a negative number, which is impossible since it is a value in the range from 0 to 1. After some debugging, it was deduced that the function was prone to integer overflow, and so the algorithm was amended to use longs instead of integers.

As well as computing the time-series for the global clustering coefficient of the network, we also gathered other data such as the in-degrees and out-degrees of nodes for exploratory analysis.

4.2 Bitcoin Price Data

The financial data that we used included price and volume data retrieved from cryptocurrency data API's such as *Cryptocator* [21], as well as websites such as *Yahoo Finance* [26]. Apart from price and volume data, we also needed volatility data, which we computed from the price data by first calculating the log-differences and then taking the absolute value of those log-differences:

$$x_v = | \log(x_p)_t^1 |$$

where x_v is the volatility and x_p corresponds to price data. This provides a good proxy for volatility for high frequency data such as daily data.

4.3 Time Series Variables

Having acquired time-series data for the clustering coefficient as well as volume and volatility, it was important to process the variables, so they could be used in our analysis. This included checking for stationarity and seasonality.

However, before going further into our methodology, it is important to discuss an early finding that affected the way we carried out our analysis (see Fig. 4.1). The figure shows the clustering coefficient and volume time-series along with the exchange rate of bitcoin for our desired time period. At roughly May 2021, a drastic regime-change in the distribution of the clustering coefficient is observed. This corresponds to the time when the exchange rate of bitcoin started to crash, and seems to be preceded by a decrease in volume. To double-check our result, we gathered data for September 2017 to September 2018, a period with similar price action to 2020-2021, with a large price increase and subsequent crash. The results were very similar, showing a drastic regime change for the clustering coefficient for the periods before and after the crash. Fig. 4.2 summarizes our results from the two time periods, showing a 70% decrease in the global clustering coefficient after the crash in 2017 and an 83% decrease in 2021.

As theorized at the beginning of our project, at times of price discovery and high speculation, the clustering of the Bitcoin network should increase. The implications of this finding are further discussed in Section 5.2, but before proceeding with our analysis, we decided it was best to split our dataset into

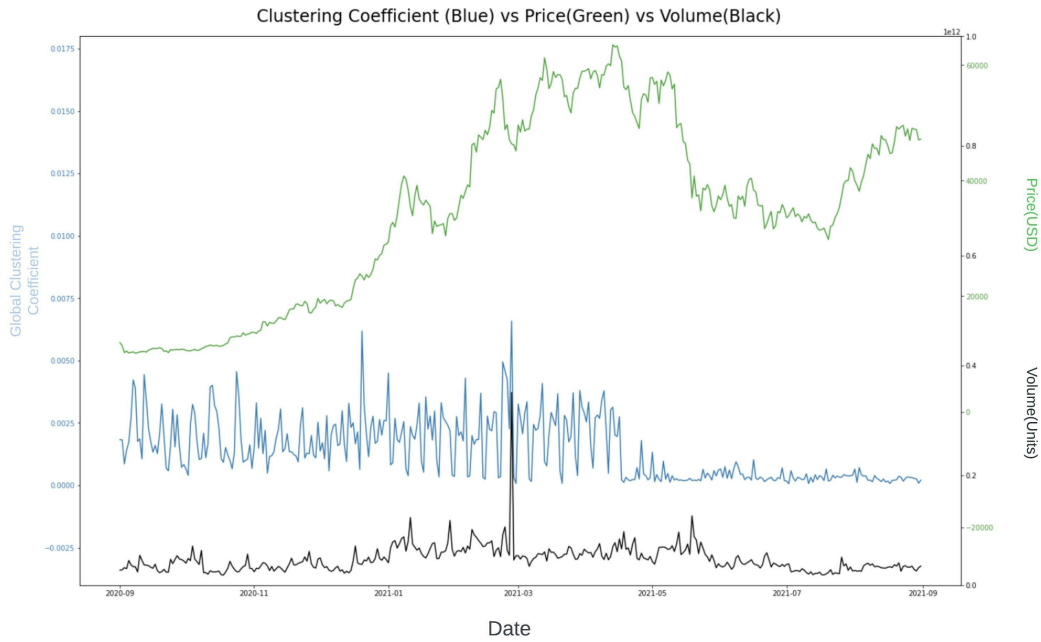


Figure 4.1: Plot of price, volume and the clustering coefficient. Notice the regime change of the clustering coefficient at 2021-5, corresponding with the crash in price.

the two regimes – one split before May 1st 2021, and one after. Hence, we would train our VAR models for each regime instead of across the two regimes. For the BUN, whose time-period was from March 2021 up to December 2021, we could not find a change in regime, so the dataset was not split.

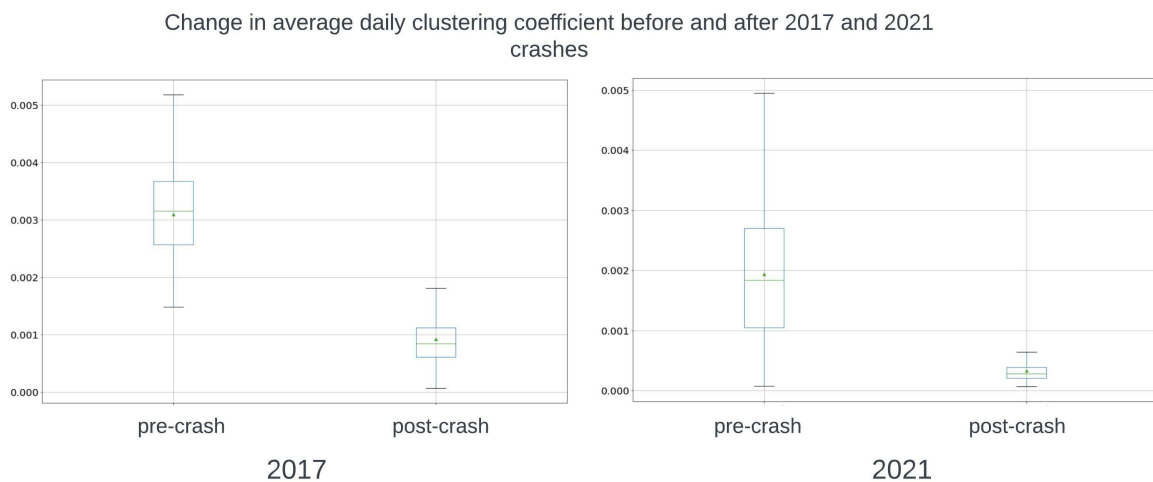


Figure 4.2: Summary of the changes to the global clustering coefficient during two identified time periods – before a crash and after a crash.

4.3.1 Checking for Stationarity In Our Variables

It is very rare that financial data is stationary, and this was the case with our time-series data. ADF tests concluded that the three variables were not stationary. Looking at the data, we could see that there was clear seasonality. In order to better understand this seasonality, we decided to have a look at the Autocorrelation Function (ACF) plots of the variables. ACF plots (also called Corelograms) are

charts of coefficients of correlation between a time-series, and its lagged values. It can sometimes be useful to identify seasonality in time-series data. Fig. 4.3 shows the ACF plot of the clustering coefficient time-series. From this plot, we can see that there is a peak roughly every 7 lags. As the data is daily, we can conclude that the data contains weekly seasonality elements. Inspection of the ACF plots for volume and volatility resulted in similar findings. This is in line with what we expected, as financial time-series very often have weekly seasonality due to occurrences such as volume decreasing on weekends.

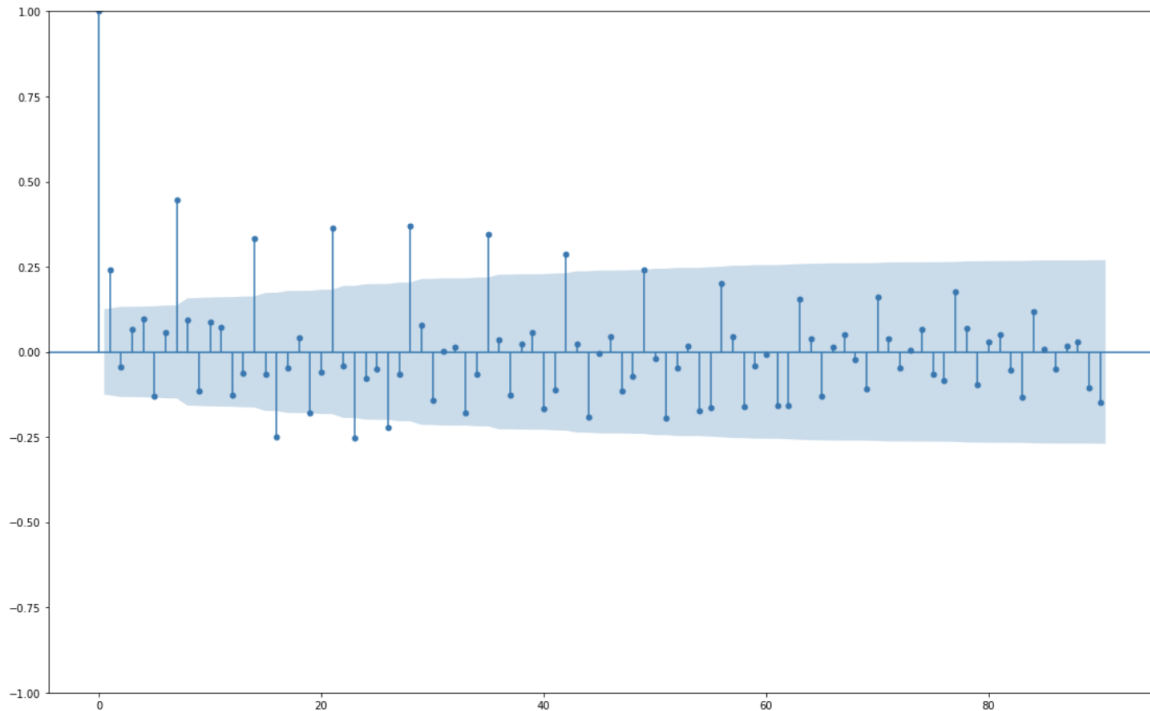


Figure 4.3: ACF plot of the global clustering coefficient time-series. Note the peaks every 7 days, indicating weekly seasonality.

Another way to double-check our seasonality assumption was by using the Python library `statsmodels`, specifically the function `seasonal_decompose`[53], which performs seasonal decomposition using moving averages.

The function decomposes a time-series into its corresponding trend and seasonality components (see Fig. 4.4). Looking at the seasonality component, we can observe that it occurs roughly four times per month, further reinforcing the evidence for weekly seasonality.

4.3.2 Making the Variables Stationary

We decided to try to make the variables stationary by first differencing at a lag of 7. This would help in removing the weekly seasonality present in the data. The equation for differencing at a lag of 7 is:

$$x_t = x_t - x_{t-7}$$

The resulting ADF tests after differencing showed that the volatility and volume time-series had become stationary, but the clustering coefficient time-series was still not stationary. Therefore, we decided to difference the data one more time at a lag of 1. Hence, the data has now been differenced twice, and the resulting ADF tests for all three time-series showed they were now stationary.

4.4 VAR Models

With our data collected and processed, it was time to build the VAR models. The plan was to construct two different models. The first model would include just the volume and volatility variables and would act as a control. We will refer to this as the *two-variable model*. The second model would include all three variables, namely volume, volatility and global clustering coefficient. We will refer to this as the



Figure 4.4: Result of seasonal decomposition of the global clustering coefficient time-series using Python’s `seasonal_decompose`. Note the seasonal component repeats 4 times per month, indicating weekly seasonality.

three-variable model. The reason we use a control is that we do not want to identify volume-volatility correlation and mistake it for clustering coefficient-volatility correlation in our three-variable model. Our desired results were that we will be able to show that the three-variable model is significantly better at predicting volatility.

Our plan was to first fit a VAR model to our whole dataset, and use the resulting VAR model to extract information about the relationships of our variables. We would then use cross-validation in order to train and test our VAR models, and compare the performance of the two-variable model with that of the three-variable model.

Selecting the VAR Model Lag

As discussed in Section 2.4.3 the method to select the optimal lag for a VAR model is to fit the model for different lags and compare some statistics produced by the model. The statistics produced, also called Information Criteria (IC), should be minimized. For this project, we decided to use the Bayesian Information Criterion (BIC) as the statistic to be minimized. When fitting the VAR model to our whole dataset, the optimal lag for both models was found to be 8, hence we used a VAR(8) model.

Cross-Validation

After fitting our VAR models to the whole dataset for some initial exploratory analysis, we decided to perform cross-validation in order to test our models and compare their performance. Cross-validation is a re-sampling method that uses different parts of the data for training and testing of a model on different iterations. The goal of cross-validation is to test a models’ ability to predict new data that was not used in training of the model. One widely used form of cross-validation, called *k-fold* cross-validation, randomly splits a dataset into k different sub-samples. One of those samples is used for testing and validation, while the rest $k - 1$ samples are used for training. The process is then repeated using a different sub-sample for validation.

For time-series, however, randomly splitting a dataset into sub-samples makes little sense, since training and evaluation sub-samples need to be in correct chronological order. The method we use is similar to *k-fold* cross validation, but is more nuanced in the selection of train and test splits. We first select a consecutive sample from the beginning of our data for training (large enough so that the training set is reliable), and use a small sample of proceeding data for testing. The test sample is then added to the original training dataset, and a new test set is formed from the next set of data points. This procedure is repeated until the end of the dataset, each iteration having a larger training set than the previous. A visualization of this method is shown in Fig. 4.5.

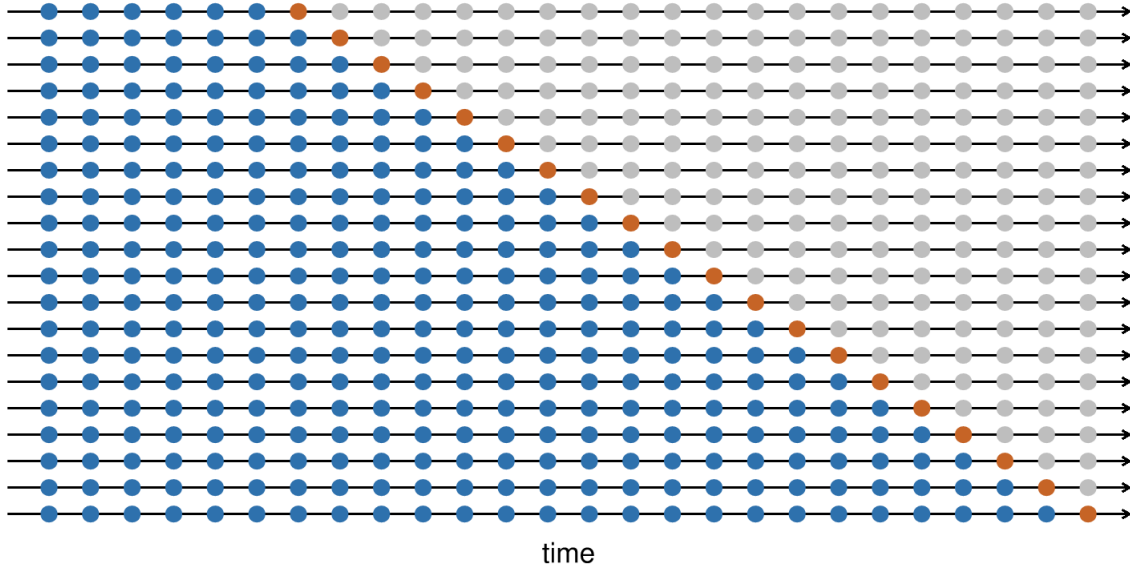


Figure 4.5: Cross-validation for time-series data. The first iteration (top) contains a train/test split such that the training set contains enough data to be considered reliable. For each subsequent iteration, the previous test set is added to the training set and a new test set is used (image taken from [44]).

For our cross-validation method, we used two different test set sizes, these being 1 day and 7 days, as we wanted results for both short and medium-term forecasting horizons.

Comparing the Performance of the VAR Models

With cross-validation set up, we could now train and test our models on the dataset and compare their forecasting performance. An important note is that since the input variables were differenced, the forecasts would be at the scale of the inputs. Therefore, to bring them back to their original scale, we have to de-difference the forecasts from the VAR models. De-differencing the forecast values proved to be a much harder task than anticipated. Information about de-differencing a time-series that has been differenced once at a lag of 1 was widely available, but I couldn't find a general algorithm that could be used for more complex cases such as ours. Recall that for our variables, we performed differencing at a lag of 7, followed by differencing again at a lag of 1. After a lot of work and testing, I created an algorithm specific to de-differencing our forecast data. The algorithm originally worked in two stages, the first stage de-differencing the second difference of lag 1, and the first stage de-differencing the first difference of lag 7. After some work, it was then made to combine the de-differencing steps into a single step, undoing both differences in one go. The final algorithm is show in Algorithm 4.1.

```

Data: train_data, forecast_data
results  $\leftarrow []$ 
i  $\leftarrow 7$ 
while  $i \leq \text{length}(\text{forecast\_data}) + 7$  do
  if  $i - 7 < 7$  then
    results[ $i - 7$ ] =
      train_data[-1] - train_data[-8] + train_data[ $i - 14$ ] + forecast_data.cumsum()[ $i - 7$ ];
  end
  else
    results[ $i - 7$ ] =
      train_data[-1] - train_data[-8] + results[ $i - 14$ ] + forecast_data.cumsum()[ $i - 7$ ];
  end
  i  $\leftarrow i + 1$ 
end

```

Algorithm 4.1: De-Differencing Algorithm

The *.cumsum()* is the cumulative sum of the forecasts up to the index specified in the square brackets. *forecast_data* are the resulting forecasts from the VAR model, and *train_data* are the non-differenced values

directly preceding the dates of the forecast data. The *results* array holds the de-differenced forecasts we are after.

The algorithm works very specifically for data that has been differenced at a lag of 7, followed by a difference at a lag of 1, but can be amended to support differences of arbitrary lags.

With forecasts now on the right scale, we were able to measure the performance of the VAR models and compare them with each other. We chose to measure the Root Mean Square Error (RMSE) as a measure of performance, as it provides more explicit information about how much predictions deviate on average from the ground truth than something like the R^2 score. The formula for the RMSE is:

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

where $\hat{y}_i$ is the predicted value, y is the actual value and n is the number of data points.

The RMSE was plotted for each cross-validation iteration, and plots of the RMSE values across all iterations were created for both models in order to compare performance.

This process was carried out for each of the regimes we identified (before May 1_{st} 2021, and after May 1_{st} 2021).

Chapter 5

Results

5.1 Exploratory Analysis

This short section will present our results regarding some general findings about the structure of the Bitcoin transaction network.

5.1.1 Degree Distributions

Both the in-degree and out-degree distributions were found to be heavy-tailed distributions. This means that in general, the transaction graph contains many low connectivity nodes, and a few nodes with very large connectivity, often called hubs.

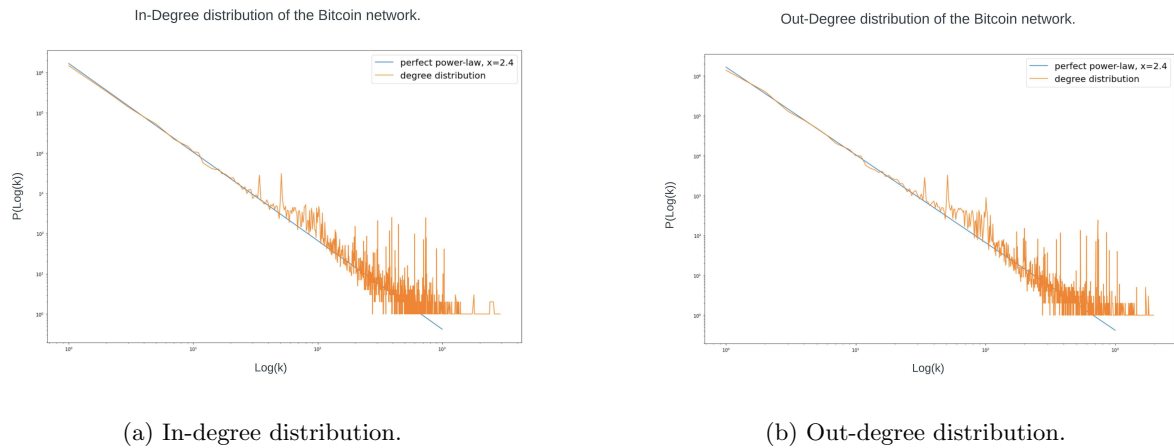


Figure 5.1: Degree distributions of the Bitcoin transaction network

Inspection of the form of the degree distributions shown in Fig. 5.1 suggests that they follow a power-law. The functional form of a power-law is:

$$y = ax^b$$

where a is a constant, and b is the power-law's exponent. When b is negative, we get what is sometimes called an inverse power-law, and it is the type of distribution that many real-world networks follows, such as the Internet and many social networks. The exponent of the power-law best fit function was found to be (-2.4) . A network with a power-law degree distribution with an exponent in the range $2 \leq b \leq 3$ is often considered to be a *scale-free* network.

A lot of research agrees with these results [56, 12, 35], even though it is the case that for some snapshots of the Bitcoin network in said research, the power-law fit appears to be less ideal than for other snapshots. Still, there is lots of debate about whether Bitcoin is indeed a scale-free network, and more generally for the definition and rarity of true scale-free networks in the real world [13].

5.1.2 Block visualizations

This section intends to showcase what a single block in the Bitcoin transaction network looks like when visualized at the address level (BAN).

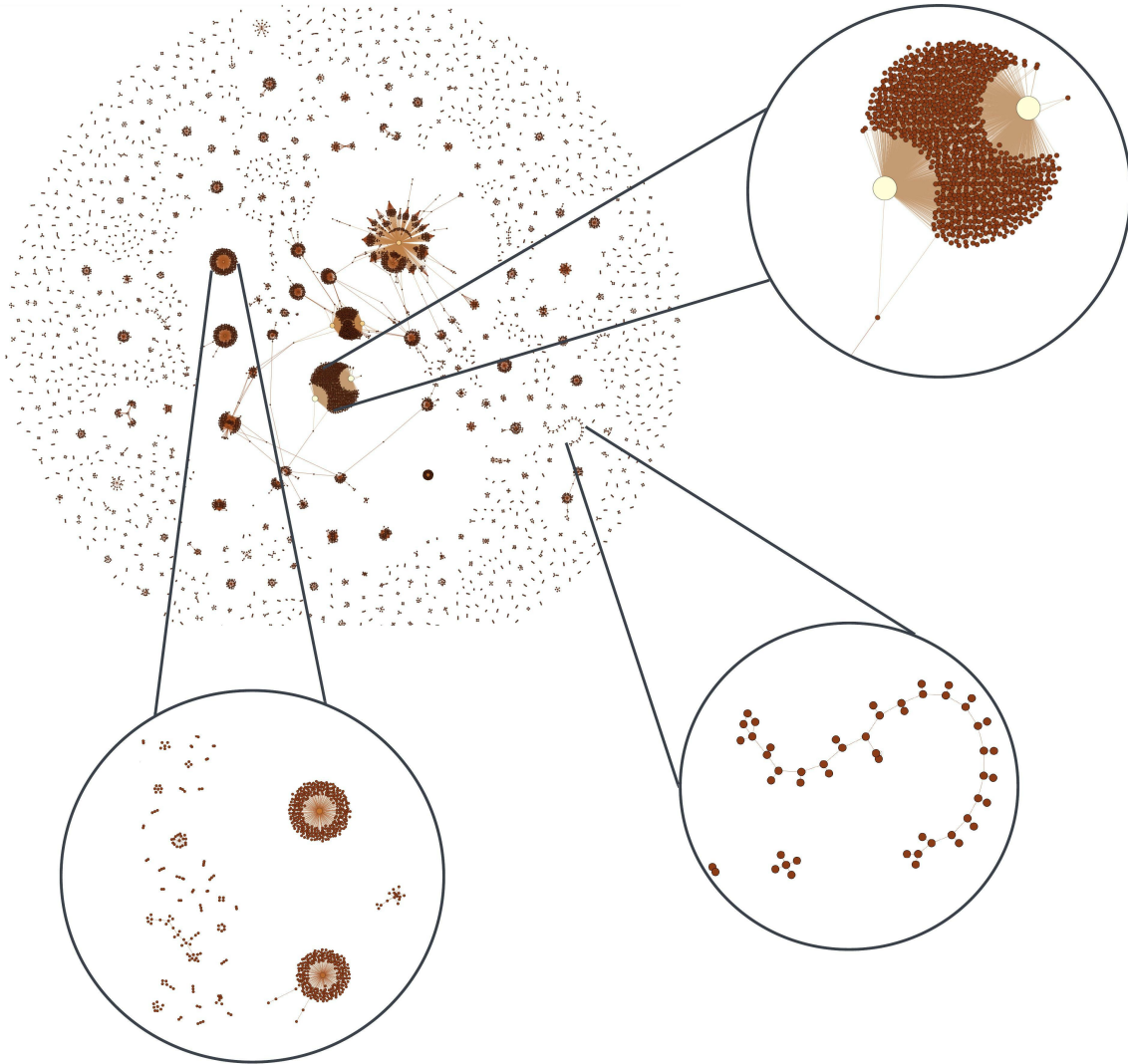


Figure 5.2: Visualization of block at height 720,000 at the address level.

The first thing that is obvious when inspecting the visualization in Fig. 5.2 is that the large majority of nodes have very few connections, but there are a few nodes with hundreds of connections. This is in line with our findings in Section 5.1.1, which state that the degree distributions are heavy-tailed distributions. Some large hubs from the network are highlighted. Although the data provided to me about which addresses belong to the nodes isn't always accurate, it would be interesting to explore further and label these hubs as perhaps belonging to exchanges or mining pools.

Also visible in the visualization (lower right highlighted part) is a chain of transactions that resembles a *peeling chain*. A peeling chain is when an address, which begins with a relatively large amount of bitcoin, sends a small amount to another address, and the remaining bitcoin is sent to a one-time change address. This process then repeats for an arbitrary number of hops, thus forming a chain of transactions. Peeling chains were a common method of laundering money and hence their detection formed part of the tracking and identification of illegal activities, but recently it is commonly used for things like exchange transactions and salary payments.

5.2 Regime Change

Before discussing our result from the VAR models, it is suitable to first talk about the regime change we observed in Section 4.3. To recap, we found that the clustering coefficient tends to be much higher during times of “bubbles”, and then drops significantly during market crashes. The duality of the two regimes means that we can potentially develop a model to identify market conditions based on the clustering coefficient. Ideally, a model would be created that would use clustering coefficient data, and be able to recognize whether we are in a bull or bear market. A bull market can be thought of as a period where prices are rising and are expected to keep rising. In cryptocurrencies, these are periods where huge increases in prices can be seen, and are identified by high amounts of speculation. A bear market is the opposite, and is a period of time when prices experience prolonged declines, usually coupled with a decrease in volume. In cryptocurrencies, drawdowns for large-cap cryptocurrencies such as Bitcoin and Ethereum can reach 80%, while for smaller altcoins the drawdowns can be upwards of 90%.

A model that can accurately predict the state of the market can be a very useful tool for traders and analysts. We believe that by using the global clustering coefficient, we can build a model that can make decisions about what state the market is in – a bullish or bearish market environment? Although such a model was not part of the original scope of this project and hence was not explored, we theorize that further work can be done based on this result, and this is discussed in Section 6.

5.3 VAR Models

The results for the BAN (Sept. 1_{st} 2020 – Aug 31_{st}. 2021) are split into two, corresponding to the two identified regimes (before May 1_{st} 2021, and after May 1_{st} 2021). The results for the BUN (March 1_{st} 2021 – Dec. 31_{st} 2021) are not split, as no regime change was identified.

5.3.1 BAN - Prior to May 1_{st} 2021

Analysing the VAR Output

The output from our fitted VAR model on the three-variable model using the whole dataset provided us with some interesting results. Fig 5.3 shows part of the VAR output for the volatility equation (see Appendix A.2.1 for full output).

The table tells us that the clustering coefficient is significant in predicting volatility at lags 1 and 8 (see the prob column, $p < 0.05$ for *L1.clust_coef* and *L8.clust_coef*). The equation for volatility resulting from this model would then be:

$$\begin{aligned}\delta\sigma = & 5.78L1.\delta C - 0.77L1.\delta\sigma - 0.64L2.\delta\sigma \\ & -0.55L3.\delta\sigma - 0.44L4.\delta\sigma - 0.38L5.\delta\sigma \\ & -0.50L7.\delta\sigma + 5.0L8.\delta C - 0.22L8.\delta\sigma\end{aligned}\tag{5.1}$$

where we use σ to denote volatility, and C to denote the clustering coefficient. The use of δ is because the variables were differenced. Of course, the VAR model output is a system of equations that form a linear dynamic system, so all three equations are needed to fully describe the VAR model (see Appendix A.2.2 for all three equations).

Although a promising initial result, it doesn’t mean much on its own and further analysis is needed before it is considered a positive result.

Adjusted R^2 Scores

The VAR model provides adjusted R^2 scores to measure the fit for each variable using the in-sample data (Table 5.1). Although the interpretation of an R^2 score depends on the field of research and type of experiment, the score of 0.64 for volatility achieved by the three-variable model is considered a moderate correlation and hence a positive result. However, when compared to the two-variable model, whose score was 0.63, the scores were not significantly different. As this does not use out-of-sample data, however, it does not say much about the performance of either model yet.

Residuals

An important bit of information provided by the fitted VAR models is information about the residuals. The residuals represent the difference between the regression line and the data points. If the relationships

Results for equation Volatility				
	coefficient	std. error	t-stat	prob
const	0.000560	0.002357	0.238	0.812
L1.clust_coef	5.781382	2.427797	2.381	0.017
L1.Volatility	-0.773829	0.072355	-10.695	0.000
L1.Volume	-0.000000	0.000000	-0.989	0.323
L2.clust_coef	-2.588777	2.349721	-1.102	0.271
L2.Volatility	-0.642387	0.088315	-7.274	0.000
L2.Volume	0.000000	0.000000	0.391	0.696
L3.clust_coef	3.856769	2.523821	1.528	0.126
L3.Volatility	-0.545742	0.101187	-5.393	0.000
L3.Volume	0.000000	0.000000	0.589	0.556
L4.clust_coef	0.567460	2.607690	0.218	0.828
L4.Volatility	-0.441007	0.106868	-4.127	0.000
L4.Volume	0.000000	0.000000	0.147	0.883
L5.clust_coef	-1.438686	2.616757	-0.550	0.582
L5.Volatility	-0.376281	0.108539	-3.467	0.001
L5.Volume	0.000000	0.000000	0.176	0.860
L6.clust_coef	2.537432	2.514510	1.009	0.313
L6.Volatility	-0.086339	0.103991	-0.830	0.406
L6.Volume	-0.000000	0.000000	-0.398	0.691
L7.clust_coef	-0.881236	2.279473	-0.387	0.699
L7.Volatility	-0.495658	0.090736	-5.463	0.000
L7.Volume	0.000000	0.000000	0.000	1.000
L8.clust_coef	4.995371	2.383959	2.095	0.036
L8.Volatility	-0.226113	0.073304	-3.085	0.002
L8.Volume	-0.000000	0.000000	-1.525	0.127

Figure 5.3: VAR output for the three-variable model for volatility.

Table 5.1: R^2 scores for the two models measured using in-sample data. The row in bold represents the variable we are interested in - volatility.

Variable	Three-variable model	Two-variable model (control)
<i>Clustering Coefficient</i>	0.50	N/A
<i>Volatility</i>	0.64	0.63
<i>Volume</i>	0.50	0.51

between the variables are linear (which is an assumption made when using VAR models), then the residuals should represent random white noise. Fig. 5.4 shows scatter plots corresponding to the residuals of the clustering coefficient and volatility. Observation of the produced scatter plots shows that the residuals don't have an underlying pattern. The scatter plot for volume, which is not shown in Fig. 5.4 also showed a lack of underlying patterns. This is a positive result that tells us that the relationship between the variables in our VAR model is linear. If there was an underlying pattern in the residuals, it would mean that our model is biased and the data is non-linear.

To further cement this linearity, we use the Durbin Watson (DW) statistic. The DW statistic is a test statistic that tests for the presence of autocorrelation at lag 1 in the residuals from a regression analysis. The DW test produces a value in the range from 0 to 4, with a value of 2 meaning there is no autocorrelation, a value less than 2 indicating positive autocorrelation, and a value greater than 2 indicating negative autocorrelation. As a rule of thumb, a DW statistic in the range from 1.5 to 2.5 is desired. The statistics produced from the residuals of our models (Table 5.2) were all very close to 2, so we were happy that there wasn't any underlying pattern, and that our linear assumption of the data and hence use of VAR was justified.

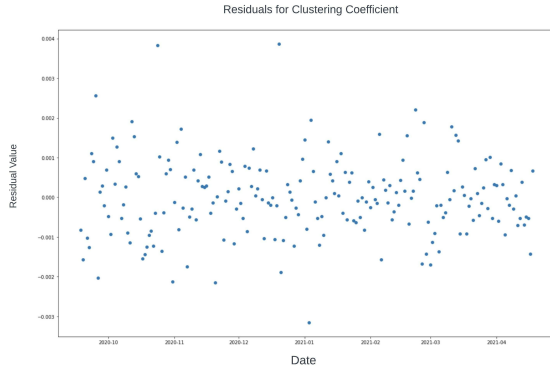
Model Performance Comparison

With the residuals checked, we were now able to compare the results of the two models. As mentioned in Section 4.4, we used cross-validation to perform training and testing, and for each iteration we computed the RMSE value for each model. We then created plots of the RMSE values, along with their mean values across all iterations. During testing, we used 1 and 7 days of forecasting.

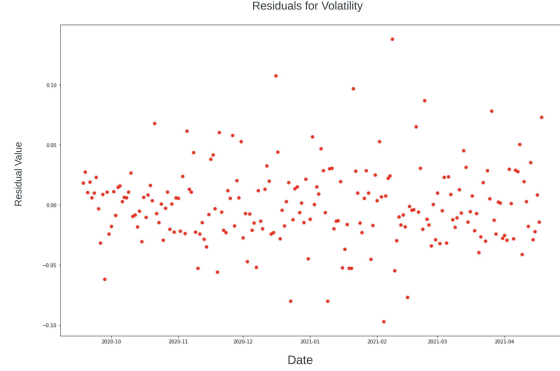
Fig. 5.5 shows the plots for the RMSE of the two models across the iterations of cross-validation. The

Table 5.2: DW test scores for each variable. As all values are very close to 2, it shows there is no autocorrelation at lag 1 in the residuals.

Variable	Three-variable model	Two-variable model (control)
<i>Clustering Coefficient</i>	2.02	N/A
<i>Volatility</i>	2.02	2.07
<i>Volume</i>	2.06	1.99



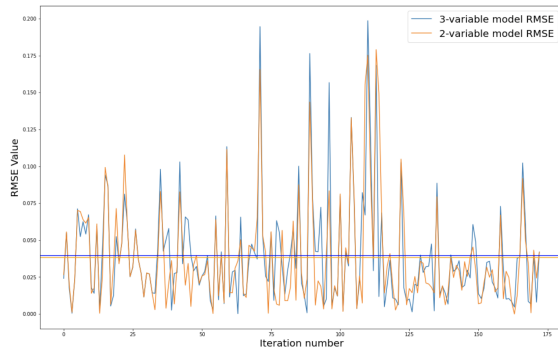
(a) Residuals for clustering coefficient



(b) Residuals for volatility

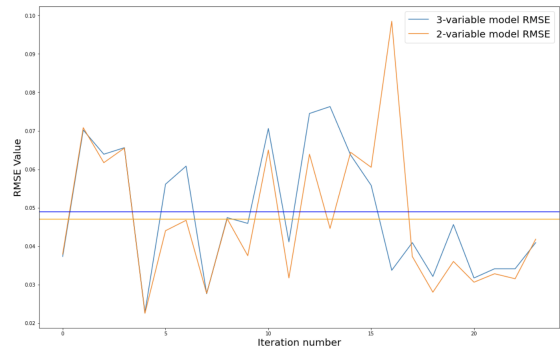
Figure 5.4: Scatter plots of the residuals from the three-variable VAR model.

RMSE comparison for 1-day forecasting(dataset before May 2021). Horizontal lines represent the mean RMSE value.



(a) 1-day forecasting results

RMSE comparison for 7-day forecasting(dataset before May 2021). Horizontal lines represent the mean RMSE value.



(b) 7-day forecasting results

Figure 5.5: RMSE plots for the two models. Horizontal lines represent the means.

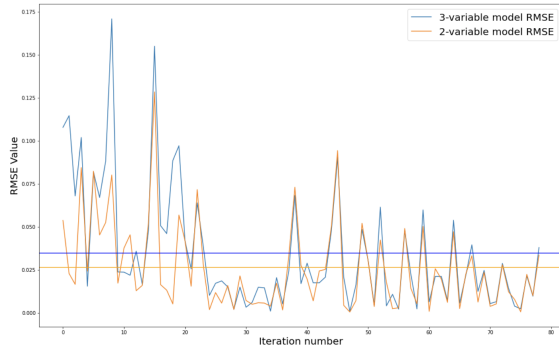
results were not what we had desired, as it clearly showed that the two models were roughly identical in their performance. Apart from the means being very close, it can be seen from the graphs that the two plots follow each other very closely. t -tests were performed on all three sets of data, and there was no evidence to suggest that we could reject the null hypothesis that the two sets of data have identical expected values.

The conclusion we draw from this analysis is that the by adding clustering coefficient data to our control containing just volume and volatility, we were unable to find any significant increase in performance.

5.3.2 BAN - After May 1st 2021

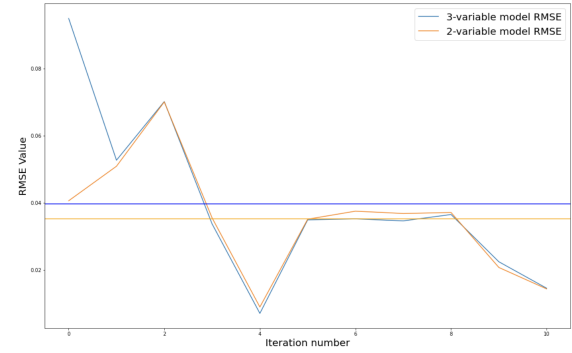
For the dataset corresponding to the second regime, we aimed to do the exact same analysis as with the prior dataset. Similarly to the previous VAR models, we first fitted a model to the whole dataset to carry out some exploratory data. Output from the fitted VAR model (see Appendix A.3.1) led to the following

RMSE comparison for 1-day forecasting(dataset after May 2021). Horizontal lines represent the mean RMSE value.



(a) 1-day forecasting results

RMSE comparison for 7-day forecasting(dataset after May 2021). Horizontal lines represent the mean RMSE value.



(b) 7-day forecasting results

Figure 5.6: RMSE plots for the two models. Horizontal lines represent the means.

equation for volatility (see Appendix A.3.2 for full system of equations):

$$\begin{aligned} \delta\sigma = & 1.0L1.\delta\sigma + 0.000037L1.\delta V - 0.95L2.\delta\sigma \\ & + 0.000052L2.\delta V - 0.65L3.\delta\sigma \end{aligned} \quad (5.2)$$

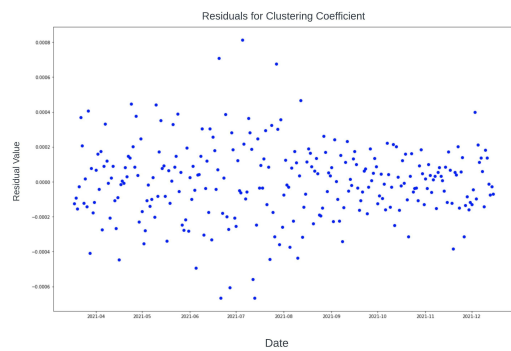
where σ stands for volatility and V stands for volume.

Similarly to the prior models, the residuals showed no underlying pattern either, and the DW test produced statistics very close to 2. Fig. 5.6 shows the plots for the RMSE values of the two models. Like the dataset prior to May 1st 2021, the two models are very similar in their forecasting performance, and we cannot conclude that the addition of the clustering coefficient to the control VAR model results in a significant increase in performance.

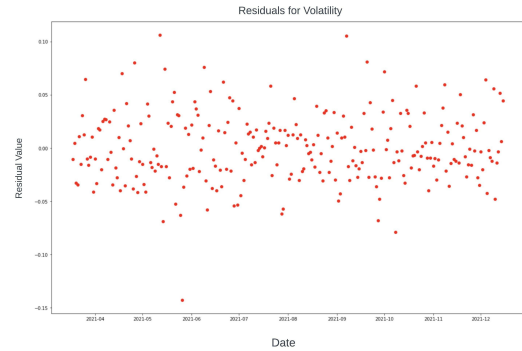
5.3.3 BUN

It was our hope that by using the BUN, which is a more elaborate version of the transaction graph than the BAN, we would be more successful in finding a significant relationship between the clustering coefficient and volatility.

The output from the VAR model, as well as the equations for the three variables, can be found in Appendix A.4. Our analysis of the resulting VAR models found confluence with the analysis from the BAN with regard to the residuals and the linearity of the relationships between the variables, and hence we were satisfied with the use of a VAR model without then need for a machine-learning model to capture non-linear relationships. The residuals for volatility and the clustering coefficient variables are shown in Fig. 5.7, while results from the DW test are summarized in Table 5.3. Values are all very close to 2 indicating a linear relationship between the variables.



(a) Residuals for clustering coefficient



(b) Residuals for volatility

Figure 5.7: Scatter plots of the residuals from the three-variable VAR model.

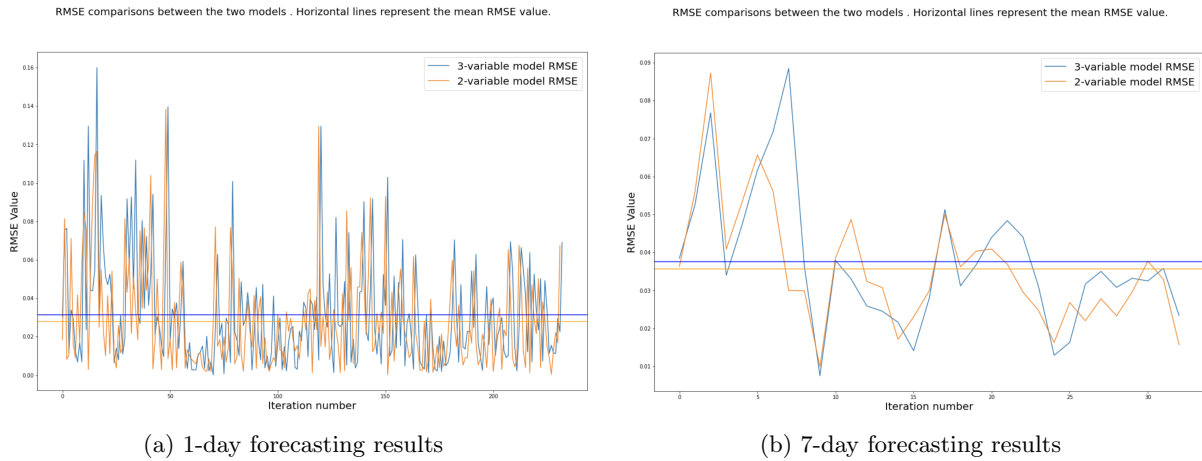


Figure 5.8: RMSE plots for the two models. Horizontal lines represent the means.

Table 5.3: DW test scores for each variable in the VAR model.

Variable	Three-variable model	Two-variable model (control)
<i>Clustering Coefficient</i>	2.08	N/A
<i>Volatility</i>	2.04	2.03
<i>Volume</i>	2.06	2.07

The resulting performances for the two VAR models however gave us near identical results to those we got from the BAN. It was obvious by looking at the RMSE plots (see Fig. 5.8) that the three-variable model did not produce a significantly better forecasting performance than the control.

5.3.4 Summary of Results

Our results from all our models are summarized in Table 5.4. We can conclude from our findings that there is insufficient evidence to suggest that the global clustering coefficient can be used to predict short-term volatility any better than volume can (at the daily time frame).

Table 5.4: RMSE scores across tests for the two models. Results in bold indicate better performance.

<i>Model</i>	<i>Three-variable model</i>	<i>Two-variable model (control)</i>
<i>BAN (before May 2021)</i>		
1-day forecasting	0.039	0.038
<i>BAN (before May 2021)</i>		
7-day forecasting	0.049	0.048
<i>BAN (after May 2021)</i>		
1-day forecasting	0.035	0.026
<i>BAN (after May 2021)</i>		
7-day forecasting	0.040	0.035
<i>BUN</i>		
1-day forecasting	0.031	0.028
<i>BUN</i>		
7-day forecasting	0.038	0.036
<i>Average</i>	0.039	0.035

Chapter 6

Conclusion

In this project, we presented our findings and analysis of the Bitcoin transaction network. Our initial goal was to use the transaction network to compute a time-series of the global clustering coefficient, and determine whether it can be used in combination with bitcoin financial data to accurately forecast volatility. In Section 4 we discussed how we carried out our experiments, from processing raw blockchain data into building the transaction graphs and then computing network properties. We decided to use two VAR models for our analysis, where one would act as a control and would contain only volume and volatility data, while the second model would also include the global clustering coefficient data. Our aim was to show a significant increase in performance between the three-variable and two-variable models, which would allow us to conclude that the clustering coefficient is a good predictor of volatility and could then be used to create forecasting models and early warning indicators. In Section 5, we presented the results of our experiments, which can be summarized as follows:

1. The global clustering coefficient is observed to undergo radical regime-changes during bubbles and crashes. During the bitcoin crashes in 2017 and 2021, the average daily clustering coefficient dropped by 70% and 83% respectively.
2. The three-variable VAR model that included the clustering coefficient was not significantly better at forecasting volatility than the control model. Hence, there is insufficient evidence to support a claim that the clustering coefficient of the Bitcoin transaction network can be used to predict short-term volatility.

As a whole, we have met the goals that we set out to accomplish in this project. Even though the outcome regarding the forecasting power of the clustering coefficient was a negative one, we are content with our methods and are pleased we continued to scrutinize the initial positive results from the VAR model (recall in Section 5.3.1, the clustering coefficient was found to be significant in predicting volatility at certain lags) to uncover that it wasn't a significant result in the bigger picture after all.

Moreover, outcome 1 (clustering coefficient regime change), which is independent of the VAR analysis we performed, is a very interesting find and lays the groundwork for further work and research.

6.1 Future Work

In this section, we want to identify and discuss four main avenues for future work that build on the results from this project.

6.1.1 Regime Change

As discussed, the finding regarding the regime change in the clustering coefficient is an important one and warrants further exploration. It is our belief that based on these findings, a model can be created that utilizes the global clustering coefficient to make a decision about the state of the market. The model would work on recent data about the clustering coefficient and would be able to inform users about whether the environment is bullish or bearish. The model could be expanded such that instead of just a binary decision (bullish, bearish), it could place the market conditions on a spectrum, ranging from one extreme to the other.

One way this can be accomplished is by using Change Point Detection (CPD) algorithms. CPD algorithms look for abrupt or structural changes in the distribution of data, and numerous implementations

exist, many of which are summarized and compared in [7, 14]. A successful model that utilizes CPD could be used to alert users of a potential change in the market environment, and can act as an early warning indicator for bubbles and crashes. Such a tool can prove to be an invaluable for analysts, traders, and researchers alike.

6.1.2 Higher Time-Frame VAR Analysis

The conclusions we made in this project regarding the use of the clustering coefficient to forecast volatility were that there isn't sufficient evidence to suggest that it is a good predictor. However, we theorized that since the clustering coefficient has been shown to potentially be a good early warning indicator for bubbles and crashes, then it might be better utilized as a predictor **across** regimes. More specifically, we should use a corresponding frequency and time-frame that is appropriate for the timescale over which bubbles and crashes occur. Therefore, it would be interesting if future work could perform similar analysis to what we have performed in this project but at a lower frequency for much larger periods of time, for example monthly data over the period of several years. At such low frequency data, volatility metrics are also more accurate (recall we used a proxy for volatility) since they can be computed by looking at the standard deviation of daily data and scaling to our frequency.

6.1.3 Individual Transaction Clustering Coefficients

Another aspect of our work that can be explored further is the transaction graph itself. More specifically, it would be insightful if a closer look at the transaction graph could be taken, for inspection of individual transactions. At the address level, single transactions can have a lot of structure, i.e. they can have hundreds of connections and patterns of inputs/outputs. For example, it is a possibility that throughout our project, our analysis was picking up information and characteristics about the nature of transaction structures rather than the actual flow of bitcoin. An example of such a characteristic could be the ratio of inputs to outputs, and it could be that that ratio is what is important in the address-level graph, and would not be directly captured by the global clustering coefficient. Therefore, it is important to determine whether it is the properties of the transaction graph that are important, or the properties of transactions themselves.

One way to go about this more nuanced analysis is by computing the clustering coefficient of individual transactions (recall that data about all transactions is available) instead of computing the global clustering coefficient for the whole transaction network. Using the computed transaction clustering coefficients, we could then show how the mean and variance of this metric changes over different time periods, and instead of the global clustering coefficient, we could instead have the average transaction clustering coefficient. VAR analysis similar to our work could then be done in order to see if the average transaction clustering coefficient is better at predicting volatility than the global clustering coefficient.

6.1.4 Network Motif Analysis

Network motifs are small sub-graphs that reflect local topology and can be utilized to uncover features and characteristics of a network. Although initially used for the analysis of large networks such as food webs and gene networks [47], it was also used in economics [46] to explain the efficiency of the price discovery process. The study focused on triadic motifs, which are sub-graphs consisting of three nodes. By measuring the proportion of certain triadic motifs in the transaction graph, we might be able to find a relationship between the proportion of said motifs, and the behaviour of bitcoin price and volatility. As these motifs capture the specific interactions at the level of traders, it would need to be performed on the BUN.

Bibliography

- [1] Coinjoin, <https://coinjoin.io/en>.
- [2] Gephi, <https://gephi.org/>.
- [3] statsmodel python package, <https://www.statsmodels.org/stable/index.html>.
- [4] Nashirah Abu Bakar and Sofian Rosbi. Autoregressive integrated moving average (arima) model for forecasting cryptocurrency exchange rate in high volatility environment: A new insight of bitcoin transaction. *International Journal of Advanced Engineering Research and Scinece*, 4, 11 2017.
- [5] A S Ahmar, A Rahman, and U Mulbar. -sutte indicator: a new method for time series forecasting. *Journal of Physics: Conference Series*, 1040:012018, jun 2018.
- [6] Moncef Amara and Amar Siad. Elliptic curve cryptography and its applications. In *International Workshop on Systems, Signal Processing and their Applications, WOSSPA*, pages 247–250, 2011.
- [7] Cook DJ. Aminikhanghahi S. Survey of methods for time series change point detection. 2017.
- [8] Apache. Spark graphx, <https://spark.apache.org/graphx/>.
- [9] Elbeck Baek, C. Bitcoins as an investment or speculative vehicle? 2015.
- [10] BBC. Cryptolocker, <https://www.bbc.co.uk/news/technology-25506020>.
- [11] Blockchain.com. Blockchain explorer.,<https://www.blockchain.com/btc/tx/3281f96f3c458a4bee6248d6667c45e8481f5>
- [12] Alexandre Bovet, Carlo Campajola, Francesco Mottes, Valerio Restocchi, Nicolò Vallarano, Tiziano Squartini, and Claudio J. Tessone. The evolving liaisons between the transaction networks of bitcoin and its price dynamics, 2019.
- [13] Anna D. Brodio and Aarong Clauset. Scale-free networks are rare. 2019.
- [14] Gerrit J. J. van den Burg and Christopher K. I. Williams. An evaluation of change point detection algorithms, 2020.
- [15] Rhonda Bush and Soohyun Choi. Forecasting ethereum storj token prices: Comparative analyses of applied bitcoin models. In *2019 International Conference on Data Mining Workshops (ICDMW)*, pages 216–223, 2019.
- [16] Vitalik Buterin. A next generation smart contract & decentralized application platform. 2015.
- [17] Y Charts. Bitcoin statistics, https://ycharts.com/indicators/reports/bitcoin_statistics.
- [18] CNBC. Top economists stiglitz, roubini and rogoﬀ renew bitcoin doom scenarios., <https://www.cnbc.com/2018/07/09/nobel-prize-winning-economist-joseph-stiglitz-criticizes-bitcoin.html>.
- [19] Thomas Conlon and Richard McGee. Safe haven or risky hazard? bitcoin during the covid-19 bear market. *Finance Research Letters*, 35:101607, 2020.
- [20] Christian Conrad, Anessa Custovic, and Eric Ghysels. Long- and short-term cryptocurrency volatility components: A garch-midas analysis. *Journal of Risk and Financial Management*, 11(2), 2018.
- [21] Cryptonator. Cryptonator, <https://www.cryptonator.com/api>.

- [22] Garcia David and Schweitzer Frank. Social signals and algorithmic trading of bitcoin. 2015.
- [23] Vasily Derbentsev, Natalia Datsenko, Olga Stepanenko, and Vitaly Bezkorovainyi. Forecasting cryptocurrency prices time series using machine learning approach. *SHS Web of Conferences*, 65:02001, 01 2019.
- [24] R Documentation. Australian beed production, <https://www.rdocumentation.org/packages/fpp/versions/0.5/topics/>
- [25] R Documentation. vars package, <https://www.rdocumentation.org/packages/vars/versions/1.5-6>.
- [26] Yahoo Finance. Btc historical data, <https://finance.yahoo.com/quote/btc-usd/history/>.
- [27] Torrent Freak. Bittorrent traffic, <https://torrentfreak.com/bittorrent-still-king-of-p2p-traffic-090218/>.
- [28] Maria Glenski, Tim Weninger, and Svitlana Volkova. Improved forecasting of cryptocurrency price using social signals, 2019.
- [29] Michael Gowan. Requiem for napster. 2002.
- [30] Martin Harrigan and Christoph Fretter. The unreasonable effectiveness of address clustering, 05 2016.
- [31] Arti Jain, Shashank Tripathi, Harsh Dhar Dwivedi, and Pranav Saxena. Forecasting price of cryptocurrencies using tweets sentiment analysis. In *2018 Eleventh International Conference on Contemporary Computing (IC3)*, pages 1–7, 2018.
- [32] Ghassan Karame. Evaluating user privacy in bitcoin. 04 2013.
- [33] Lee S.-Y.T. Kim, K. and S Assar. The dynamics of cryptocurrency market behavior: sentiment analysis using markov chains. 2022.
- [34] Dimitrios Koutmos. Bitcoin returns and transaction activity. *Economics Letters*, 167(C):81–85, 2018.
- [35] Zeng D Liang J, Li L. Evolutionary dynamics of cryptocurrency transaction networks: An empirical study, 2018.
- [36] Yukun Liu and Aleh Tsyvinski. Risks and returns of cryptocurrency. 2018.
- [37] R. East M. Malak. *Spark GraphX in Action*. 2016.
- [38] Sean McNally, Jason Roche, and Simon Caton. Predicting the price of bitcoin using machine learning. In *2018 26th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, pages 339–343, 2018.
- [39] Sarah Meiklejohn, Marjori Pomarole, Grant Jordan, Kirill Levchenko, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. A fistful of bitcoins: Characterizing payments among men with no names. In *Proceedings of the 2013 Conference on Internet Measurement Conference*, IMC ’13, page 127–140, New York, NY, USA, 2013. Association for Computing Machinery.
- [40] Malte Möser and Arvind Narayanan. Resurrecting address clustering in bitcoin, 2021.
- [41] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2009.
- [42] Solvej; Gradstein Helen Natarajan, Harish; Krause. Distributed ledger technology and blockchain. 2017.
- [43] Mark Newman. *Networks*. 2018.
- [44] otexs. time series cross validation, <https://otexs.com/fpp3/tscv.html>.
- [45] Manoj Parameswaran, Anjana Susarla, and Andrew Whinston. P2p networking: An information-sharing alternative. *Computer*, 34:31 – 38, 08 2001.
- [46] Steve Phelps and Katarzyna Musial. Network motifs for microeconomic analysis. 06 2013.

- [47] S. Itzkovitz N. Kashtan D. Chlovskii R. Milo, S. Shen-Orr and U. Alon. Network motifs: simple building blocks of complex networks. 2002.
- [48] Fergal Reid and Martin Harrigan. An analysis of anonymity in the bitcoin system, 2011.
- [49] Dorit Ron and Adi Shamir. Quantitative analysis of the full bitcoin transaction graph. 11 2012.
- [50] Tiwari A.-Hammoudeh S Selmi, R. Efficiency or speculation? a dynamic analysis of the bitcoin market. 2018.
- [51] Mariano Sorgente and Cristian Cibils. The reaction of a network: Exploring the relationship between the bitcoin network structure and the bitcoin price. 2014.
- [52] Michele Spagnuolo, Federico Maggi, and Stefano Zanero. Bitiodine: Extracting intelligence from the bitcoin network. volume 8437, pages 457–468, 03 2014.
- [53] statsmodels. seasonal decompose, https://www.statsmodels.org/dev/generated/statsmodels.tsa.seasonal.seasonal_decompose.html.
- [54] James H. Stock and Mark W. Watson. Vector autoregressions. *Journal of Economic Perspectives - Volume 15*, 2001.
- [55] Dian Utami Sutiksno, Ansari Saleh Ahmar, Nuning Kurniasih, Eko Susanto, and Audrey Leiwakabessy. Forecasting historical data of bitcoin using ARIMA and -sutte indicator. *Journal of Physics: Conference Series*, 1028:012194, jun 2018.
- [56] Nicolás Vallarano, Claudio J. Tessone, and Tiziano Squartini. Bitcoin transaction networks: An overview of recent results. *Frontiers in Physics*, 8, 2020.
- [57] Işıl Yenidoğan, Aykut Çayır, Ozan Kozan, Tuğçe Dağ, and Çiğdem Arslan. Bitcoin forecasting using arima and prophet. pages 621–624, 2018.

Appendix A

A.1 Global Clustering Coefficient Algorithm

```
def clusteringCoefficient[VD:ClassTag,ED:ClassTag](g:Graph[VD,ED]) = {  
  val numTriplets = g.aggregateMessages[Set[VertexId]](  
    et => { et.sendToSrc(Set(et.dstId));  
            et.sendToDst(Set(et.srcId)) },  
    (a,b) => a ++ b  
    .map(x => {val s = (x._2 - x._1).size; s*(s-1) / 2})  
    .reduce(_ + _)  
  )  
  if (numTriplets == 0) 0.0 else  
    g.triangleCount.vertices.map(_._2).reduce(_ + _) /  
    numTriplets.toFloat  
}
```

Listing A.1: Function to compute the global clustering coefficient of a graph in Apache Spark. Language is Scala (taken from *Spark GraphX in Action*, M. Malak, R. East) [37]

This function calculates the global clustering coefficient by computing the total number of triplets in the network, and dividing by the total number of triangles (closed triplets). For the number of triplets, the variable *numTriplets* is used. The built-in function *aggregateMessages* is a function that collects information about neighbouring nodes for each node in the network. In this case, each node collects the node id's of its neighbours. Sets are used to collect the id's so that duplicates and hence parallel edges are removed. Once we have all the neighbouring nodes for each vertex, computing the total number of triplets for a vertex is a combinatoric combination of n choose 2:

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

For the total number of triangles, the built-in method *TriangleCount* is used.

A.2 BAN VAR Output - Prior to May 1_{st} 2021

A.2.1 Output

This is the resulting output from the VAR fitted to the whole dataset.

Summary of Regression Results				
=====				
Model:	VAR			
Method:	OLS			
Date:	Wed, 04, May, 2022			
Time:	16:39:47			
=====				
No. of Equations:	3.00000	BIC:	29.2911	
Nobs:	213.000	HQIC:	28.5859	
Log likelihood:	-3825.16	FPE:	1.61576e+12	
AIC:	28.1076	Det(Omega_mle):	1.15820e+12	
=====				
Results for equation clust_coef				
=====				
	coefficient	std. error	t-stat	prob
const	-0.000012	0.000071	-0.172	0.864
L1.clust_coef	-0.569185	0.072936	-7.804	0.000
L1.Volatility	-0.001921	0.002174	-0.884	0.377
L1.Volume	-0.000000	0.000000	-0.668	0.504
L2.clust_coef	-0.394315	0.070590	-5.586	0.000
L2.Volatility	-0.002151	0.002653	-0.811	0.417
L2.Volume	-0.000000	0.000000	-1.093	0.275
L3.clust_coef	-0.434446	0.075821	-5.730	0.000
L3.Volatility	0.001509	0.003040	0.496	0.620
L3.Volume	0.000000	0.000000	0.052	0.959
L4.clust_coef	-0.270736	0.078340	-3.456	0.001
L4.Volatility	0.007962	0.003211	2.480	0.013
L4.Volume	0.000000	0.000000	0.102	0.919
L5.clust_coef	-0.296308	0.078613	-3.769	0.000
L5.Volatility	0.007276	0.003261	2.231	0.026
L5.Volume	-0.000000	0.000000	-0.391	0.696
L6.clust_coef	-0.154483	0.075541	-2.045	0.041
L6.Volatility	0.006617	0.003124	2.118	0.034
L6.Volume	-0.000000	0.000000	-0.428	0.668
L7.clust_coef	-0.561297	0.068480	-8.197	0.000
L7.Volatility	0.005555	0.002726	2.038	0.042
L7.Volume	-0.000000	0.000000	-0.504	0.614
L8.clust_coef	-0.223588	0.071619	-3.122	0.002
L8.Volatility	0.001848	0.002202	0.839	0.401
L8.Volume	-0.000000	0.000000	-0.082	0.935
=====				
Results for equation Volatility				
=====				
	coefficient	std. error	t-stat	prob
const	0.000560	0.002357	0.238	0.812
L1.clust_coef	5.781382	2.427797	2.381	0.017
L1.Volatility	-0.773829	0.072355	-10.695	0.000
L1.Volume	-0.000000	0.000000	-0.989	0.323
L2.clust_coef	-2.588777	2.349721	-1.102	0.271
L2.Volatility	-0.642387	0.088315	-7.274	0.000
L2.Volume	0.000000	0.000000	0.391	0.696
L3.clust_coef	3.856769	2.523821	1.528	0.126
L3.Volatility	-0.545742	0.101187	-5.393	0.000
L3.Volume	0.000000	0.000000	0.589	0.556
L4.clust_coef	0.567460	2.607690	0.218	0.828
L4.Volatility	-0.441007	0.106868	-4.127	0.000
L4.Volume	0.000000	0.000000	0.147	0.883
L5.clust_coef	-1.438686	2.616757	-0.550	0.582
L5.Volatility	-0.376281	0.108539	-3.467	0.001
L5.Volume	0.000000	0.000000	0.176	0.860
L6.clust_coef	2.537432	2.514510	1.009	0.313
L6.Volatility	-0.086339	0.103991	-0.830	0.406
L6.Volume	-0.000000	0.000000	-0.398	0.691
L7.clust_coef	-0.881236	2.279473	-0.387	0.699
L7.Volatility	-0.495658	0.090736	-5.463	0.000
L7.Volume	0.000000	0.000000	0.000	1.000
L8.clust_coef	4.995371	2.383959	2.095	0.036
L8.Volatility	-0.226113	0.073304	-3.085	0.002
L8.Volume	-0.000000	0.000000	-1.525	0.127

Figure A.1: VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.

Results for equation Volume			
prob	coefficient	std. error	t-stat
const	396042709.788438	2160828567.378179	0.183
L1.clust_coef	-2315877477517.718262	2225945595770.231445	-1.040
L1.Volatility	84252862238.848846	66339235895.563660	1.270
L1.Volume	-0.710835	0.073350	-9.691
L2.clust_coef	1275451554242.645752	2154360342069.104980	0.592
L2.Volatility	38682708710.231880	80972204580.077789	0.478
L2.Volume	-0.467026	0.084185	-5.548
L3.clust_coef	3304528799091.873047	2313985529236.147949	1.428
L3.Volatility	137174693685.842270	92774366568.764832	1.479
L3.Volume	-0.291760	0.091498	-3.189
L4.clust_coef	3880815623821.170898	2390882082221.771973	1.623
L4.Volatility	147307463195.687592	97982850991.245499	1.503
L4.Volume	-0.145837	0.093087	-1.567
L5.clust_coef	3376469354256.622559	2399194771032.836426	1.407
L5.Volatility	105650231129.138489	99514705224.280167	1.062
L5.Volume	-0.093696	0.091433	-1.025
L6.clust_coef	19062445781.070087	2305448460265.018066	0.008
L6.Volatility	61935829117.937645	95345020475.368027	0.650
L6.Volume	-0.021973	0.087001	-0.253
L7.clust_coef	-944072258504.917236	2089953673585.791992	-0.452
L7.Volatility	56060145482.443245	83191788756.051102	0.674
L7.Volume	-0.418457	0.080614	-5.191
L8.clust_coef	-1467253066006.188721	2185751711932.147461	-0.671
L8.Volatility	5002922119.024655	67209223904.502037	0.074
L8.Volume	-0.258453	0.069867	-3.699
Correlation matrix of residuals			
	clust_coef	Volatility	Volume
clust_coef	1.000000	-0.058027	0.175594
Volatility	-0.058027	1.000000	0.191721
Volume	0.175594	0.191721	1.000000

Figure A.2: CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.

A.2.2 Equations

$$\begin{aligned}\delta\sigma &= 5.78L1.\delta C - 0.77L1.\delta\sigma - 0.64L2.\delta\sigma \\ &\quad - 0.55L3.\delta\sigma - 0.44L4.\delta\sigma - 0.38L5.\delta\sigma \\ &\quad - 0.50L7.\delta\sigma + 5.0L8.\delta C - 0.22L8.\delta\sigma\end{aligned}\tag{A.1}$$

$$\begin{aligned}\delta C &= -0.57L1.\delta C - 0.39L2.\delta C - 0.43L3.\delta C \\ &\quad - 0.27L4.\delta C + 0.0080L4.\delta\sigma - 0.30L5.\delta C \\ &\quad + 0.0073L5.\delta\sigma - 0.15L6.\delta C + 0.0066L6.\delta\sigma \\ &\quad - 0.56L7.\delta C + 0.0056L7.\delta\sigma - 0.22L8.\delta C\end{aligned}\tag{A.2}$$

$$\begin{aligned}\delta V &= -0.71L1.\delta V - 0.47L2.\delta V - 0.29L3.\delta V \\ &\quad - 0.42L7.\delta V - 0.26L8.\delta V\end{aligned}\tag{A.3}$$

where σ stands for volatility, C stands for the clustering coefficient and V stands for volume.

A.3 BAN VAR Output - After May 1_{st} 2021

A.3.1 Output

This is the resulting output from the VAR fitted to the whole dataset.

Summary of Regression Results				
Model:	VAR			
Method:	OLS			
Date:	Wed, 04, May, 2022			
Time:	17:06:53			
No. of Equations:	3.00000	BIC:	25.0816	
Nobs:	94.0000	HQIC:	23.8721	
Log likelihood:	-1408.60	FPE:	1.06807e+10	
AIC:	23.0524	Det(Omega_mle):	5.26430e+09	
Results for equation clust_coef				
	coefficient	std. error	t-stat	prob
const	0.000004	0.000023	0.188	0.851
L1.clust_coef	-0.938949	0.121014	-7.759	0.000
L1.Volatility	-0.001041	0.000895	-1.162	0.245
L1.Volume	-0.000000	0.000000	-0.188	0.851
L2.clust_coef	-0.636465	0.161072	-3.951	0.000
L2.Volatility	-0.000131	0.001233	-0.106	0.915
L2.Volume	-0.000000	0.000000	-0.134	0.894
L3.clust_coef	-0.487585	0.177533	-2.746	0.006
L3.Volatility	-0.000594	0.001451	-0.409	0.682
L3.Volume	-0.000000	0.000000	-0.306	0.760
L4.clust_coef	-0.391117	0.187068	-2.091	0.037
L4.Volatility	0.000767	0.001517	0.505	0.613
L4.Volume	-0.000000	0.000000	-1.102	0.271
L5.clust_coef	-0.067687	0.184084	-0.368	0.713
L5.Volatility	0.000574	0.001507	0.381	0.703
L5.Volume	-0.000000	0.000000	-0.051	0.959
L6.clust_coef	-0.100761	0.174995	-0.576	0.565
L6.Volatility	0.000029	0.001408	0.021	0.983
L6.Volume	-0.000000	0.000000	-0.191	0.849
L7.clust_coef	-0.298585	0.160298	-1.863	0.063
L7.Volatility	0.000899	0.001166	0.771	0.440
L7.Volume	0.000000	0.000000	0.651	0.515
L8.clust_coef	-0.051508	0.120830	-0.426	0.670
L8.Volatility	-0.000473	0.000874	-0.541	0.589
L8.Volume	0.000000	0.000000	1.438	0.150
Results for equation Volatility				
	coefficient	std. error	t-stat	prob
const	-0.000979	0.003579	-0.274	0.784
L1.clust_coef	-1.922539	18.548882	-0.104	0.917
L1.Volatility	-1.006380	0.137223	-7.334	0.000
L1.Volume	0.000000	0.000000	2.926	0.003
L2.clust_coef	3.345890	24.688781	0.136	0.892
L2.Volatility	-0.953068	0.189047	-5.041	0.000
L2.Volume	0.000000	0.000000	2.442	0.015
L3.clust_coef	-22.467093	27.211972	-0.826	0.409
L3.Volatility	-0.654776	0.222473	-2.943	0.003
L3.Volume	-0.000000	0.000000	-0.104	0.917
L4.clust_coef	-17.510433	28.673480	-0.611	0.541
L4.Volatility	-0.225258	0.232544	-0.969	0.333
L4.Volume	0.000000	0.000000	0.726	0.468
L5.clust_coef	24.994898	28.216104	0.886	0.376
L5.Volatility	-0.275632	0.231055	-1.193	0.233
L5.Volume	0.000000	0.000000	0.587	0.557
L6.clust_coef	11.007394	26.823003	0.410	0.682
L6.Volatility	-0.090870	0.215865	-0.421	0.674
L6.Volume	0.000000	0.000000	0.463	0.643
L7.clust_coef	-0.919376	24.570166	-0.037	0.970
L7.Volatility	-0.298299	0.178708	-1.669	0.095
L7.Volume	-0.000000	0.000000	-0.585	0.559
L8.clust_coef	-9.386502	18.520559	-0.507	0.612
L8.Volatility	-0.082364	0.133922	-0.615	0.539
L8.Volume	-0.000000	0.000000	-1.879	0.060

Figure A.3: VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.

Results for equation Volume			
prob	coefficient	std. error	t-stat
const	-453720184.810044	1137220121.246958	-0.399
L1.clust_coef	-2547003265176.367676	5893513939097.903320	-0.432
L1.Volatility	11354258174.240532	43599633815.031136	0.260
L1.Volume	-0.326882	0.125219	-2.610
L2.clust_coef	-2734964242092.071289	7844336739260.244141	-0.349
L2.Volatility	-57989682887.568283	60065556515.612274	-0.965
L2.Volume	0.003712	0.142963	0.026
L3.clust_coef	-9174132641190.875000	8646027115466.553711	-1.061
L3.Volatility	-44162552918.801460	70686067315.344971	-0.625
L3.Volume	-0.277402	0.145306	-1.909
L4.clust_coef	-7577790778427.821289	9110390068768.142578	-0.832
L4.Volatility	18999318661.721279	73885888385.323502	0.257
L4.Volume	-0.000807	0.143991	-0.006
L5.clust_coef	1772455170044.295410	8965068781627.160156	0.198
L5.Volatility	18491755268.968491	73412850513.989319	0.252
L5.Volume	-0.112538	0.143662	-0.783
L6.clust_coef	-259150948.595340	8522440297850.609375	-0.000
L6.Volatility	-13153318195.909204	68586409438.591400	-0.192
L6.Volume	0.066650	0.141786	0.470
L7.clust_coef	6281134047038.137695	7806649070529.221680	0.805
L7.Volatility	33764309913.353275	56780557002.101212	0.595
L7.Volume	-0.342529	0.132816	-2.579
L8.clust_coef	1870031158955.688965	5884514751182.733398	0.318
L8.Volatility	62494185180.913811	42550733194.911766	1.469
L8.Volume	-0.367538	0.110216	-3.335
Correlation matrix of residuals			
	clust_coef	Volatility	Volume
clust_coef	1.000000	-0.018876	-0.097014
Volatility	-0.018876	1.000000	0.529265
Volume	-0.097014	0.529265	1.000000

Figure A.4: CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.

A.3.2 Equations

$$\begin{aligned}\delta\sigma = & 1.0L1.\delta\sigma + 0.000037L1.\delta V - 0.95L2.\delta\sigma \\ & + 0.000052L2.\delta V - 0.65L3.\delta\sigma\end{aligned}\tag{A.4}$$

$$\begin{aligned}\delta C = & -0.94L1.\delta C - 0.64L2.\delta C - 0.49L3.\delta C \\ & - 0.39L4.\delta C\end{aligned}\tag{A.5}$$

$$\delta V = -0.33L1.\delta V - 0.34L7.\delta V - 0.37L8.\delta V\tag{A.6}$$

where σ stands for volatility, C stands for the clustering coefficient and V stands for volume.

A.4 BUN VAR Output

A.4.1 Output

This is the resulting output from the VAR fitted to the whole dataset.

Summary of Regression Results

```

=====
Model:                               VAR
Method:                             OLS
Date:                               Thu, 05, May, 2022
Time:                               00:50:43
=====
No. of Equations:                    3.00000    BIC:                               23.6011
Nobs:                               273.000    HQIC:                              23.0076
Log likelihood:                      -4173.31    FPE:                               6.60479e+09
AIC:                                22.6095    Det(Omega_mle):                    5.07806e+09
=====

```

Results for equation clust_coef

```

=====
               coefficient      std. error      t-stat      prob
-----
const          -0.000000         0.000014       -0.034       0.973
L1.clust_coef  -0.473560         0.064444       -7.348       0.000
L1.Volatility   0.000439         0.000473        0.929       0.353
L1.Volume       -0.000000         0.000000       -1.997       0.046
L2.clust_coef  -0.337349         0.067301       -5.013       0.000
L2.Volatility   0.000540         0.000627        0.861       0.389
L2.Volume       -0.000000         0.000000       -2.545       0.011
L3.clust_coef  -0.093617         0.072231       -1.296       0.195
L3.Volatility   0.000227         0.000725        0.313       0.755
L3.Volume       -0.000000         0.000000       -1.208       0.227
L4.clust_coef  -0.171061         0.073657       -2.322       0.020
L4.Volatility   -0.000430         0.000770       -0.558       0.577
L4.Volume       -0.000000         0.000000       -1.347       0.178
L5.clust_coef  -0.012042         0.072869       -0.165       0.869
L5.Volatility   -0.000242         0.000769       -0.314       0.753
L5.Volume       -0.000000         0.000000       -0.334       0.738
L6.clust_coef  -0.076097         0.072613       -1.048       0.295
L6.Volatility   0.000101         0.000721        0.141       0.888
L6.Volume       -0.000000         0.000000       -0.760       0.447
L7.clust_coef  -0.393008         0.067962       -5.783       0.000
L7.Volatility   0.000487         0.000624        0.781       0.435
L7.Volume       0.000000         0.000000        0.457       0.648
L8.clust_coef  -0.147877         0.064009       -2.310       0.021
L8.Volatility   -0.000037         0.000473       -0.077       0.938
L8.Volume       -0.000000         0.000000       -0.227       0.820
=====

```

Results for equation Volatility

```

=====
               coefficient      std. error      t-stat      prob
-----
const           0.000298         0.002058        0.145       0.885
L1.clust_coef   10.546479         9.790430        1.077       0.281
L1.Volatility   -0.935985         0.071784      -13.039       0.000
L1.Volume       0.000000         0.000000        1.656       0.098
L2.clust_coef   11.014425        10.224452        1.077       0.281
L2.Volatility   -0.804151         0.095316       -8.437       0.000
L2.Volume       0.000000         0.000000        2.099       0.036
L3.clust_coef   -6.113215        10.973386       -0.557       0.577
L3.Volatility   -0.686424         0.110118       -6.234       0.000
L3.Volume       0.000000         0.000000        1.164       0.245
L4.clust_coef    5.117316        11.190036        0.457       0.647
L4.Volatility   -0.401345         0.116994       -3.430       0.001
L4.Volume       0.000000         0.000000        1.232       0.218
L5.clust_coef   12.379858        11.070288        1.118       0.263
L5.Volatility   -0.251376         0.116894       -2.150       0.032
L5.Volume       0.000000         0.000000        0.200       0.841
L6.clust_coef   21.186914        11.031387        1.921       0.055
L6.Volatility   -0.123401         0.109544       -1.126       0.260
L6.Volume       0.000000         0.000000        0.333       0.739
L7.clust_coef    7.309934        10.324886        0.708       0.479
L7.Volatility   -0.483560         0.094832       -5.099       0.000
L7.Volume       0.000000         0.000000        0.449       0.653
L8.clust_coef    6.315900         9.724298        0.649       0.516
L8.Volatility   -0.203099         0.071808       -2.828       0.005
L8.Volume       0.000000         0.000000        0.099       0.921
=====

```

Figure A.5: VAR output for the three-variable model for volatility. Shows the tables for the clustering coefficient and volatility. Remaining output is on the next page.

Results for equation Volume

	coefficient	std. error	t-stat	prob
const	-48542494.254847	677876419.128982	-0.072	0.943
L1.clust_coef	-3490691363451.881836	3224115158135.568359	-1.083	0.279
L1.Volatility	18879926256.778355	23639537685.600700	0.799	0.424
L1.Volume	-0.502082	0.072907	-6.887	0.000
L2.clust_coef	-6546770247675.540039	3367044236035.575684	-1.944	0.052
L2.Volatility	-26419426781.052208	31388881697.366234	-0.842	0.400
L2.Volume	-0.233652	0.080661	-2.897	0.004
L3.clust_coef	-10062505051867.925781	3613677852903.888672	-2.785	0.005
L3.Volatility	-58596979748.364189	36263199866.340508	-1.616	0.106
L3.Volume	-0.215700	0.085926	-2.510	0.012
L4.clust_coef	-5836978930144.333008	3685023468565.077148	-1.584	0.113
L4.Volatility	-45856375484.044380	38527525716.828377	-1.190	0.234
L4.Volume	0.004523	0.086706	0.052	0.958
L5.clust_coef	-7594022962502.702148	3645588971410.261719	-2.083	0.037
L5.Volatility	-30321859369.282291	38494645616.604324	-0.788	0.431
L5.Volume	-0.146695	0.086369	-1.698	0.089
L6.clust_coef	-736593675423.637085	3632778289358.722656	-0.203	0.839
L6.Volatility	-46268616500.768303	36074355779.885727	-1.283	0.200
L6.Volume	0.017456	0.083996	0.208	0.835
L7.clust_coef	-1967298667059.573975	3400118222515.760742	-0.579	0.563
L7.Volatility	-28192263029.152485	31229436786.638229	-0.903	0.367
L7.Volume	-0.380981	0.077517	-4.915	0.000
L8.clust_coef	-603904341246.234741	3202336996041.520020	-0.189	0.850
L8.Volatility	16141380291.600941	23647214689.311188	0.683	0.495
L8.Volume	-0.238831	0.070216	-3.401	0.001

Correlation matrix of residuals

	clust_coef	Volatility	Volume
clust_coef	1.000000	-0.104767	-0.230043
Volatility	-0.104767	1.000000	0.509139
Volume	-0.230043	0.509139	1.000000

Figure A.6: CONTINUED: VAR output for the three-variable model for volatility. Shows the table for volume.

A.4.2 Equations

$$\begin{aligned}\delta\sigma = & -0.94L1.\delta\sigma - 0.80L2.\delta\sigma + 0.0000095L2.\delta V - 0.69L3.\delta\sigma \\ & -0.40L4.\delta\sigma - 0.25L5.\delta\sigma - 0.48L7.\delta\sigma - 0.20L8.\delta\sigma\end{aligned}\tag{A.7}$$

$$\begin{aligned}\delta C = & -0.47L1.\delta C - 0.00000031L1.\delta V - 0.34L2.\delta C - 0.00000081L2.\delta V \\ & -0.17L4.\delta C - 0.39L7.\delta C - 0.15L8.\delta C\end{aligned}\tag{A.8}$$

$$\begin{aligned}\delta V = & -0.50L1.\delta V - 0.23L2.\delta V - 0.22L3.\delta V - 100L3.\delta C \\ & -76L5.\delta C - 0.15L5.\delta V - 0.38L7.\delta V - 0.24L8.\delta V\end{aligned}\tag{A.9}$$

where σ stands for volatility, C stands for the clustering coefficient and V stands for volume.